



**COMSATS University Islamabad,
Sahiwal Campus**

Reflect – Personalized AI Learning Assistant

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Software Engineering (2022-2026)

By

Muhammad Khuram CIIT/SP22-BSE-028/SWL

Asadullah CIIT/SP22-BSE-010/SWL

Sibgha Babar CIIT/SP22-BSE-041/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning

Muhammad Khuram

Asadullah

Sibgha Babar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) “**Reflect – Personalized AI Learning Assistant**” was developed by **Muhammad Khuram (CIIT/SP22-BSE-028/SWL)**, **Asadullah (CIIT/SP22-BSE-010/SWL)** and **Sibgha Babar (CIIT/SP22-BSE-041/SWL)** under the supervision of “**Mr. Fahad Amin**” and ” and co supervisor “**Mr. Ahamad Shaf**” that and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Mr. Fahad Amin

Professor

Department of Software engineering
COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Shafiq Hussain

Associate Professor

University of Sahiwal

Head of Department

Dr. Zafar Iqbal Roy

Associate Professor

Department of computer science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In the modern educational landscape, traditional e-learning platforms often fail to address the unique needs of individual learners. Most existing solutions, such as Coursera or basic coding bootcamps, utilize a "one-size-fits-all" approach where static content is delivered regardless of a student's proficiency or learning pace. This leads to a lack of engagement, inefficient learning paths, and a disconnect between theoretical knowledge and practical application.

"Reflect – Personalized AI Learning Assistant" addresses these critical gaps by leveraging Artificial Intelligence (AI) to create a dynamic, adaptive, and personalized learning experience. Unlike static platforms, Reflect utilizes the Groq API (Generative AI) to analyze user performance in real-time, generating customized learning roadmaps and adapting quiz difficulty on the fly.

This project encompasses the full Software Development Life Cycle (SDLC), employing the MERN stack (MongoDB, Express.js, React.js, Node.js, Next.js) for a scalable web architecture. The system includes modules for User Management, AI-Powered Roadmap Generation, Real-Time Code Execution, and Gamified Progress Tracking (Future). This report details the comprehensive design, implementation, and testing of Reflect, demonstrating its potential to revolutionize coding education by treating every student as a unique learner.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. Fahad Amin**” and our and co-supervisor “**Mr. Ahmad Shaf**”. Without his supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Khuram

Asadullah

Sibgha Babar

Abbreviations

API	Application Programming Interface
UI	User Interface
SRS	Software Require Specification
OTP	One-Time Password
AI	Artificial Intelligence
UML	Unified Modeling Language

Table of Contents

Contents

1. Introduction.....	12
1.1 Brief Overview	12
1.2 Relevance to Course Modules	12
1.2.1 Technical Skills:.....	12
1.2.2 Software Engineering Principles:	13
1.2.3 Database Design:.....	13
1.2.4 System Architecture:.....	13
1.2.5 UI/UX Design:	13
1.2.6 Project Management:	13
1.2.7 Networking:.....	13
1.3 Project Background	14
1.4 Literature Review	14
1.6 Methodology and Software Lifecycle	15
1.6.1 Design Methodology: Object-Oriented Design (OOD).....	15
2. Problem Statement.....	18
2.1 The Proposed Solution: Reflect	19
2.2 Deliverables and Development Requirements.....	19
2.2.1 Core Deliverables	19
2.2.2 Development Requirements.....	19
3. Requirement Identifying Technique	22
3.1 Use Case Diagram	22
3.1.1 User Registration and Login	23
3.1.2 Profile Management	24
3.1.4 Personalized Roadmap Generation	25
3.1.5 Quiz Attempt & Feedback.....	26
3.1.6 Final Course Test	27
3.1.7 Leaderboard and Progress Tracking	28
3.1.8 Certification and Badges	29
3.1.9 Roadmap Progress Update	30
3.1.10 Manual Quiz and Challenge Creation	31
3.1.11 User Performance Monitoring.....	32
3.1.12 Content Management	32
3.1.13 Leaderboard and Achievement Management	33
3.1.14 AI-Based Quiz Generation.....	34
3.1.15 AI-Based Learning Recommendations.....	34

3.1.16 Code Execution and Evaluation	35
3.1.17 AI-Generated Final Test Questions	36
3.2 Functional Requirements	37
3.2.1 User Registration and Login	37
3.2.2 Profile Management	38
3.2.3 Personalized Learning Roadmap	38
3.2.4 Quiz Attempt and Feedback	39
3.2.5 Coding Challenge Submission and Execution	39
3.2.6 Progress Tracking	40
3.2.7 AI-Based Learning Recommendations.....	40
3.3 Non-Functional Requirements	42
3.3.1 Usability Requirements	42
3.3.2 Performance Requirements	43
3.3.3 Reliability and Availability Requirements.....	43
3.3.4 Security Requirements	43
3.3.5 Maintainability and Scalability Requirements.....	44
3.3.6 Compatibility Requirements	44
4. Design and Architecture	47
4.1 System Architecture.....	47
4.2 Data Representation	48
4.3 Process Flow / Representation	50
4.4 Class Diagram	52
4.5 Sequence Diagram	53
5. Implementation	55
5.1 Frontend Development	55
5.2 Backend Development	55
5.3 Integration of AI Algorithms	56
5.4 User Interface.....	56
6. Testing and Evaluation.....	67
6.1 Manual Testing.....	67
6.2 Unit Testing	68
6.3 Functional Testing	69
6.4 Integration Testing.....	69
7. Conclusion and Future Work	72
7.1 Conclusion	72
7.2 Future Work.....	72
8.References.....	73

List of Figures

Figure 3.1 Reflect System – Use Case Diagram.....	22
Figure 4.1 : Three-tier architecture Diagram.....	48
Figure 4.2 : Process Flow Representation.....	51

List of Tables

Table 3.1: Use Case - User Registration and Login	23
Table 3.2: Use Case - Profile Management.....	24
Table 3.3: Use Case - Logout.....	25
Table 3.4: Use Case - Personalized Roadmap Generation	26
Table 3.5: Use Case - Quiz Attempt and Feedback.....	27
Table 3.6: Use Case - Final Course Test	28
Table 3.7: Use Case - Leaderboard and Progress Tracking	29
Table 3.8: Use Case - Certification & Badges.....	29
Table 3.9: Use Case - Roadmap Progress Update.....	30
Table 3.10: Use Case - Manual Quiz and Challenge Creation.....	31
Table 3.11: Use Case - User Performance Monitoring	32
Table 3.12: Use Case - Content Management.....	32
Table 3.13: Use Case - Leaderboard and Achievement Management	33
Table 3.14: Use Case - AI-Based Quiz Generation	34
Table 3.15: Use Case - AI-Based Learning Recommendations.....	35
Table 3.16: Use Case - Code Execution and Evaluation.....	35
Table 3.17: Use Case - AI-Generated Final Test Questions	36
Table 3.18: Functional Requirement - User Registration and Login	37
Table 3.19: Functional Requirement - Profile Management	38
Table 3.20: Functional Requirement - Personalized Learning Roadmap	38
Table 3.21: Functional Requirement - Quiz Attempt and Feedback	39
Table 3.22: Functional Requirement - Coding Challenge Submission and Execution	39
Table 3.23: Functional Requirement - Progress Tracking.....	40
Table 3.24: Functional Requirement - AI-Based Learning Recommendations	40
Table 3.25: Functional Requirement - Code Execution and API Integration	41
Table 6.1 : Manual Testing	67
Table 6.2 : Unit Testing	68
Table 6.3 : Functional Testing	69
Table 6.4 : Integration Testing.....	70

Chapter 1

Introduction

1. Introduction

Reflect – Personalized AI Learning Assistant is an advanced educational platform that merges the structured curriculum of a Learning Management System (LMS) with the adaptability of modern Artificial Intelligence. The primary objective of this project is to create an environment where the course content adjusts itself to the student, rather than forcing the student to adjust to the content. The system operates by first assessing the user's current skill level through an initial evaluation. Based on this data, the system leverages the Groq API to generate a structured JSON-based roadmap. As the user progresses through topics ranging from basic variables to complex algorithms the system continually assesses their understanding through AI-generated quizzes and practical coding challenges. Unlike static platforms, Reflect provides instant, context-aware feedback. If a student fails a quiz on "Loops," the system does not simply mark it wrong; it generates a simplified explanation and suggests a prerequisite topic to review. This dynamic remediation loop is the core innovation of the project.

1.1 Brief Overview

Reflect is a web-based application that serves as a comprehensive coding tutor.

1. **User-Centric Design:** It offers a clean, distraction free interface for learning.
2. **AI Integration:** It uses Large Language Models (LLMs) to generate educational content on the fly, ensuring that no two learning paths are exactly the same.
3. **Practical Application:** It includes a full featured Integrated Development Environment (IDE) in the browser, supporting multiple languages (Python, JavaScript, C++).
4. **Gamification (Future):** It uses points, badges, and leaderboards to psychologically motivate students to maintain a daily learning streak.

1.2 Relevance to Course Modules

The design and development of Reflect required the practical application of theoretical concepts learned throughout the BS Software Engineering degree:

1.2.1 Technical Skills:

The project is built on the MERN stack. This required mastery of **JavaScript (ES6+)** for both client and server-side logic. We utilized **React.js** hooks for state management and **Redux** for global store management. The backend required deep knowledge of **Node.js** event loops and **Express.js** middleware. Database management involved designing NoSQL schemas in **MongoDB**.

1.2.2 Software Engineering Principles:

We adhered strictly to software engineering best practices. We utilized **Modular Design**, ensuring that the "Quiz Module" was decoupled from the "User Module." We employed **Version Control** (Git/GitHub) with feature-branch workflows to manage collaboration among three team members without code conflicts.

1.2.3 Database Design:

The data requirements for an adaptive system are complex. We designed a flexible schema to store "Roadmaps," which are nested tree structures. We implemented indexing on userID and quizID to optimize query performance, ensuring the dashboard loads in under 2 seconds.

1.2.4 System Architecture:

We implemented a **Service-Oriented Architecture (SOA)**. The frontend is a distinct service consuming APIs from the backend. The backend acts as an orchestrator, managing communication between the database, the AI provider (Groq).

1.2.5 UI/UX Design:

Applying Human-Computer Interaction (HCI) principles, we designed the system to minimize cognitive load. We used consistent color schemes (Dark Mode support), intuitive navigation bars, and clear feedback mechanisms (spinners, toast notifications) to keep the user informed of system status.

1.2.6 Project Management:

We utilized **Agile Scrum**. The project was divided into 4 major sprints:

1. Requirement Gathering & UI Design.
2. Core Backend & AI Integration.
3. Frontend Development.
4. Testing & Deployment.

1.2.7 Networking:

The application relies on secure data transmission. We implemented **CORS (Cross-Origin Resource Sharing)** policies to allow the frontend to communicate with the backend securely. We used **Axios interceptors** to attach JWT tokens to headers automatically for authenticated requests.

1.3 Project Background

The motivation for Reflect stems from the observation of the "Tutorial Hell" phenomenon. Novice programmers often consume hours of video content but fail to write code themselves. When they attempt to write code, they encounter errors they cannot understand. Existing solutions are polarized: some are too easy (passive video watching) and some are too hard (competitive programming sites). Reflect fills the middle ground. It provides the guidance of a video course but demands the active participation of a coding platform. By utilizing AI, we can scale the experience of having a personal tutor to thousands of students simultaneously.

1.4 Literature Review

We conducted an extensive review of existing platforms to identify the feature gaps that Reflect intends to fill.

1. **Khan Academy:** Excellent for K-12 education but lacks advanced, dynamic coding environments for university-level algorithms.
2. **LeetCode:** The industry standard for interviews, but it creates high anxiety for beginners. It provides "Test Cases" but not "Teaching."
3. **Udemy:** A marketplace for video courses. Content is often outdated quickly, and there is no interactivity. If a student has a question, they must post in a forum and wait days for a reply.
4. **ChatGPT:** While powerful, using raw ChatGPT for learning is disjointed. It does not track a user's long-term progress, nor does it have a built-in compiler to verify if the generated code actually runs.

1.5 Analysis from Literature Review

Table 1.1: Related System Analysis

Feature	LeetCode	Udemy	ChatGPT	Reflect (Proposed)
Structured Roadmap	No	Yes	No	Yes(AI Generated)
Adaptive Difficulty	No	No	Yes	Yes
Real-time Compiler	Yes	No	No	Basic

Feature	LeetCode	Udemy	ChatGPT	Reflect (Proposed)
Personalized Feedback	No	No	Yes	Yes
Gamification	Yes	No	No	Not Yet

The table clearly shows that while competitors possess individual features, no single platform combines structured learning, adaptive AI, and real-time execution. Reflect creates a unified ecosystem that addresses these shortcomings.

1.6 Methodology and Software Lifecycle

For the development of Reflect – Personalized AI Learning Assistant, we adopted a hybrid approach that integrates the Object-Oriented Design (OOD) methodology for system architecture with the Agile Scrum process model for the software development lifecycle (SDLC). This combination was selected after a careful analysis of the project's unique requirements, specifically the need to integrate unpredictable AI behaviors (via the Groq API) with deterministic code execution features (via Judge0) in future. The project is not merely a static web application; it is a dynamic system where the data structures (roadmaps, quizzes) are generated in real-time. This complexity necessitated a design philosophy that prioritizes modularity (OOD) and a process model that prioritizes adaptability (Agile).

1.6.1 Design Methodology: Object-Oriented Design (OOD)

We utilized Object-Oriented Design to structure the backend and database schemas. The OOD approach allows us to model the system as a collection of interacting objects rather than a linear set of functions. This is crucial for "Reflect" because the entities in our system Users, Roadmaps, and Quizzes have complex relationships that map well to object-oriented principles.

Key OOD Principles Applied:

- **Encapsulation:** Data related to a specific user (e.g., quizScores, completedTopics) is encapsulated within the User object. The internal logic for calculating a user's progress is hidden from the rest of the system, exposed only through specific methods like updateProgress(). This ensures data integrity and prevents accidental modification from unrelated modules.

- **Inheritance:** The system distinguishes between different user roles. We defined a base User class containing common attributes (ID, Email, Password). The Student and Admin classes inherit from this base class but extend it with role-specific methods. For example, an Admin can deleteUser(), while a Student can submitCode(). This reduces code redundancy.
- **Polymorphism:** The system handles different types of learning content (Text Lessons, Video Links, Coding Challenges) using a common interface. The ContentRenderer component can take any content object and render it appropriately, regardless of its specific type.

Chapter 2

Problem Definition

2. Problem Statement

The domain of computer science education is currently facing a significant crisis regarding engagement and retention. While the demand for software engineering skills has skyrocketed, the traditional methods of online learning have failed to keep pace with the needs of modern learners. The core problem lies in the "one-size-fits-all" approach adopted by the majority of E-Learning platforms.

1. Lack of Personalization in Traditional Curriculums:

Traditional platforms, such as Coursera, Udemy, or standard university Learning Management Systems (LMS), deliver content linearly. A novice programmer and an experienced developer looking to switch languages are forced to follow the exact same syllabus. This leads to two detrimental outcomes: the novice becomes overwhelmed by complexity and drops out, while the expert becomes bored by redundancy and disengages. There is a distinct lack of adaptive systems that can assess a user's starting point and tailor the content accordingly.

2. The "Passive Learning" Trap:

Most existing solutions rely heavily on video lectures as the primary medium of instruction. Educational psychology research suggests that passive consumption of video content yields significantly lower retention rates compared to active problem-solving. Students often fall into "Tutorial Hell," where they can follow along with a video but cannot write code independently because they lack a platform that enforces active practice.

3. Absence of Immediate, Contextual Feedback:

In a physical classroom, a teacher can look at a student's code and immediately point out a logic error. In self-paced online learning, this feedback loop is broken. Platforms like LeetCode provide "Pass/Fail" results based on hidden test cases, but they rarely explain *why* the code failed or how to fix the underlying logic. This lack of explanatory feedback causes frustration and halts the learning process.

4. Fragmentation of the Learning Ecosystem:

Currently, a student must use multiple disjointed tools to learn effectively:

- **YouTube/Udemy** for theoretical concepts.
- **LeetCode/HackerRank** for coding practice.
- **ChatGPT/StackOverflow** for debugging help.
- **Notion/Excel** for tracking progress. This fragmentation breaks the user's focus and makes it difficult to maintain a cohesive record of learning history.

2.1 The Proposed Solution: Reflect

"Reflect" aims to resolve these specific issues by creating a unified, single-window learning environment. By integrating the **Groq API**, the system addresses the personalization gap, generating unique roadmaps for every user. By integrating this, the system addresses the passive learning trap, forcing users to write and execute code in real-time, In future this will be added. The system creates a closed-loop ecosystem where instruction, practice, and assessment happen seamlessly.

2.2 Deliverables and Development Requirements

To successfully address the problems outlined above, the project aims to deliver a comprehensive web-based platform. The scope of the project is defined by the following specific deliverables:

2.2.1 Core Deliverables

1. User Authentication & Profile Module: A secure system allowing users to create accounts, manage sessions, and store their learning history. This module must support JWT-based stateless authentication to ensure scalability. It will also handle profile customization, allowing users to set their learning goals (e.g., "Frontend Development" vs. "Data Science").

2. AI-Powered Roadmap Generator: This is the flagship feature of Reflect. Unlike static database entries, this module acts as an interface between the user and the Large Language Model (LLM). It accepts user inputs regarding skill level and generates a structured JSON object representing a step-by-step study plan. This roadmap must be interactive, allowing users to click on nodes (topics) to access detailed content.

3. Intelligent Quiz Engine: A module capable of generating dynamic Multiple-Choice Questions (MCQs). The system must not rely on a fixed bank of questions. Instead, it should generate questions on-the-fly based on the specific topic the user is studying. It must also provide detailed "Why/Why Not" explanations for every answer option.

4. Gamified Dashboard: A visualization layer that tracks user progress. This includes graphical representations of completed modules, a "Streak" counter to encourage daily usage, and a global Leaderboard to introduce a competitive social element.

2.2.2 Development Requirements

The development of Reflect requires a specific set of tools and technologies to ensure high performance and maintainability.

1. Hardware Requirements:

- **Development Machine:** Minimum 8GB RAM, Core i5 Processor (for running local Node servers and Docker containers).
- **Client Device:** Any web-enabled device (Laptop, Tablet, Smartphone) with a modern browser (Chrome 90+, Firefox 88+).
- **Software & Tools:**
 - **API Testing:** Postman (for testing backend routes before frontend integration).
 - **Version Control:** Git and GitHub (for source code management and team collaboration).
 - **Design:** Figma (for creating high-fidelity wireframes and UI prototypes).
 - **Runtime Environment:** Node.js (v18.17.0 or higher).
- **External APIs:**
 - **Groq Cloud API:** For high-speed AI inference (Llama-3 model).
 - **MongoDB Atlas:** For cloud-based NoSQL database hosting.

1. Current System:

It is important to document the "Current System" (i.e., the status quo) to understand the improvement Reflect offers. Currently, students at COMSATS University and self-learners generally rely on a manual combination of disparate resources.

2. Existing Workflow:

- A student searches for a tutorial on YouTube (e.g., "Learn Python").
- They watch a 2-hour video passively.
- If they encounter an error, they paste it into Google or ChatGPT.
- They have no way to test themselves unless they find a separate quiz site.
- There is no record of their progress other than their browser history.

3. Limitations of Current System:

- **Disjointed Experience:** Context switching between video, editor, and search engine destroys focus.
- **No Accountability:** Without a progress tracker or leaderboard, motivation dwindles quickly.
- **Static Content:** If a student doesn't understand the video explanation, there is no "alternative explanation" available instantly.

Reflect replaces this broken workflow with a streamlined, all-in-one solution.

Chapter 3

Requirement Analysis

3. Requirement Identifying Technique

Use case modeling provides the foundation for identification and specification of the requirements for Reflect: An AI-Powered Personalized Learning Platform. Since Reflect is an interactive web-based program where the user does activities such as take quizzes, complete coding exercises, and view the progress, use cases are the most appropriate technique for identification of system requirements.

3.1 Use Case Diagram



Figure 3.1 Reflect System – Use Case Diagram

3.2 Use Case Descriptions

The table below indicate a comprehensive use case template filled in with an example drawn from the Reflect System.

3.1.1 User Registration and Login

The User Registration & Login use case offers the functionality for registration by new users (students and admins) and login by existing users. Users must provide basic details such as name, email, and password at the time of registration, which are verified by the system before being securely stored in the database. For security, all passwords are hashed before storage, and users can be prompted to verify their email. Users provide their credentials at the time of logging in, which are verified by the system through JWT tokens. The system raises an error message and, after multiple attempts, temporarily locks the account for security reasons if invalid credentials are provided.

Table 3.1: Use Case - User Registration and Login

Use Case ID	UC-01
Use Case Name	User Registration and Login
Actors	Student, Admin
Description	This use case allows users (students and admins) to create an account, log in, and authenticate securely.
Trigger	The user accesses the system and initiates the registration or login process.
Preconditions	The user must have a valid email address and meet password requirements.
Postconditions	The user is successfully authenticated and redirected to their dashboard.
Normal Flow	1. The user selects Sign Up or Login from the homepage. 2. For Sign Up, the user provides their name, email, and password. 3. The system validates the credentials and stores user data securely. 4. For Login, the user enters their email and password. 5. The system verifies credentials and grants access. 6. The user is redirected to their respective dashboard (Student or Admin).
Alternative Flows	1. If the user forgets their password, they can reset it via email. 2. If the email is not registered, the system notifies the user and they must create an account.
Exceptions	1. Invalid email or password results in an error message.

Use Case ID	UC-01
	2. If the server is down, the system displays a “ Service Unavailable ” message.
Business Rules	1. Passwords must be at least 8 characters with one uppercase letter, one number, and one special character. 2. Users must verify their email before logging in.
Assumptions	1. Users have a stable internet connection. 2. The email verification system is functioning properly.

3.1.2 Profile Management

Users can modify their profile details, including name, email, password, and profile picture. Students can even modify their learning preferences and skill goals, which directly affect their personalized learning plan. Admins are shown profile update features but without learning preferences. The system stores and reflects changes in the user database in real time, and the users always see the most current details.

Table 3.2: Use Case - Profile Management

Use Case ID	UC-02
Use Case Name	Profile Management
Actors	Student, Admin
Description	Allows users to update their personal information, including name, profile picture, and learning preferences.
Trigger	The user selects the Edit Profile option.
Preconditions	The user must be logged in.
Postconditions	1. The updated profile information is saved in the system.
Normal Flow	1. The user accesses the Profile page. 2. They update their information. 3. The system validates and saves the changes.
Alternative Flows	If the user does not confirm changes, the system retains the previous data.
Exceptions	If the server is down, the system notifies the user that changes could not be saved.
Business Rules	Users cannot leave required fields empty.

Use Case ID	UC-02
Assumptions	The user provides valid information.

3.1.3 Logout

Users can securely log out of their session, preventing unauthorized access.

Table 3.3: Use Case - Logout

Use Case ID	UC-03
Use Case Name	Logout
Actors	Student, Admin
Description	Ends the user session and logs them out securely.
Trigger	The user clicks the Logout button.
Preconditions	The user must be logged in.
Postconditions	The user is logged out, and their session is terminated.
Normal Flow	1. The user clicks Logout. 2. The system invalidates the session and removes authentication tokens. 3. The user is redirected to the Login Page.
Alternative Flows	If the session expires due to inactivity, the system logs out the user automatically.
Exceptions	If the server fails to log out the user, they are notified and asked to try again.
Business Rules	1. After logout, no authentication token should remain stored. 2. The system should auto-logout inactive users after 15 minutes.
Assumptions	The user has an active internet connection.

3.1.4 Personalized Roadmap Generation

After completing an initial skill assessment, the system dynamically generates a personalized learning roadmap according to the user's coding ability. The roadmap consists of structured topics, including theory lessons, quizzes, and coding practice exercises. While the users learn, the AI monitors performance and modifies the learning path accordingly. If the users struggle with a topic, the system suggests additional practice material or easier exercises before moving on to advanced levels. High-achieving users, however, can be provided with accelerated content to keep their learning effective.

Table 3.4: Use Case - Personalized Roadmap Generation

Use Case ID	UC-04
Use Case Name	Personalized Roadmap Generation
Actors	Student, System
Description	AI generates a personalized learning roadmap based on user skills, goals, and initial quiz results.
Trigger	The user completes the initial skill assessment quiz .
Preconditions	1. The user must be logged in. 2. The system must have access to AI for quiz analysis.
Postconditions	A customized roadmap is created and stored for the user.
Normal Flow	1. The user answers the initial assessment quiz . 2. The system analyzes the answers using AI. 3. The system generates a personalized roadmap with recommended topics. 4. The roadmap is displayed on the dashboard.
Alternative Flows	The user can manually adjust their roadmap if needed.
Exceptions	If AI fails to generate a roadmap, the system suggests a default structured path .
Business Rules	1. Roadmaps must align with the user's skill level. 2. Users can only reset their roadmap once every 30 days .
Assumptions	AI-powered quiz analysis is functioning properly.

3.1.5 Quiz Attempt & Feedback

Users may attempt admin-constructed or AI-generated quizzes as part of the study plan. Quizzes consist of multiple-choice questions, and users select answers within a predefined time limit. Upon submission, the system evaluates answers, calculates the score, and provides detailed feedback for wrong answers. Users receive explanations for wrong answers and study recommendations. In addition, quiz attempts are also part of overall progress tracking to allow users to understand strong and weak areas.

Table 3.5: Use Case - Quiz Attempt and Feedback

Use Case ID	UC-05
Use Case Name	Quiz Attempt & Feedback
Actors	Student, System
Description	Users take AI-generated quizzes to test knowledge and receive feedback.
Trigger	The user selects "Take Quiz" for a specific topic.
Preconditions	1. The user must have an active learning roadmap. 2. The system must have AI-generated quiz questions available.
Postconditions	1. The user receives a score and feedback on weak areas.
Normal Flow	1. The user selects a quiz. 2. The system loads AI-generated questions. 3. The user answers the questions. 4. The system evaluates the responses and provides instant feedback . 5. The user sees their score and topic suggestions for improvement.
Alternative Flows	If AI fails to generate new questions, the system provides pre-saved quiz questions .
Exceptions	If there is a server error, the quiz cannot be loaded. The system notifies the user.
Business Rules	Users must complete at least 80% of quizzes to unlock the Final Course Test.
Assumptions	The system has a sufficient database of quiz questions.

3.1.6 Final Course Test

After completing a learning plan, students must pass a final exam, which tests everything learned. The test includes multiple-choice questions, coding assignments, and theory descriptions, ensuring a comprehensive evaluation of the user's skills. Difficulty is automatically adjusted by AI based on past performance, ensuring fairness and applicability. Certification and achievement badges are awarded to students for a passing score, enhancing motivation and skill validation.

Table 3.6: Use Case - Final Course Test

Use Case ID	UC-06
Use Case Name	Final Course Test
Actors	Student, System
Description	Users take a comprehensive test before completing a course. The test includes quizzes and coding challenges.
Trigger	The user completes at least 80% of the learning roadmap .
Preconditions	1. The user must have completed most quizzes and coding challenges . 2. The system must have AI-generated questions ready.
Postconditions	The system evaluates the test, assigns a score, and awards a certificate if passed.
Normal Flow	1. The user selects "Take Final Test" . 2. The system loads AI-generated MCQs and a coding problem . 3. The user answers the MCQs and submits code. 4. The system evaluates the answers and provides a score. 5. If the user passes, they receive a certificate and leaderboard update .
Alternative Flows	If the user fails, they can retake the test after revising weak topics .
Exceptions	If AI fails to generate questions, the system provides pre-saved test questions .
Business Rules	1. Users must score at least 70% to pass. 2. Users can take a maximum of 3 attempts per course.
Assumptions	The system generates unique test questions for each user.

3.1.7 Leaderboard and Progress Tracking

Reflect follows a gamified learning approach, and the users earn points for quiz scores, coding challenge completion, and milestone achievement. The students are ranked on a leaderboard based on points earned, encouraging competition and motivation. The progress tracking is done dynamically by the system, and the users are provided with real-time metrics on their learning process.

Table 3.7: Use Case - Leaderboard and Progress Tracking

Use Case ID	UC-07
Use Case Name	Leaderboard and Progress Tracking
Actors	Student, System, Admin
Description	Displays user rankings based on quiz scores, coding challenges, and final test performance. Allows users to track their learning progress.
Trigger	The user accesses the Leaderboard or Progress Dashboard .
Preconditions	The user must have attempted at least one quiz or coding challenge.
Postconditions	The leaderboard is updated, and user progress is recorded.
Normal Flow	1. The user views the Leaderboard or Progress Dashboard . 2. The system retrieves user scores and ranks from the database. 3. The system displays ranking, points, and progress stats .
Alternative Flows	If no leaderboard data exists, the system displays a “ Start Learning to Earn Points ” message.
Exceptions	If the system cannot fetch scores, it displays an error message.
Business Rules	1. Only completed quizzes and challenges count towards leaderboard points. 2. Users must be active in the last 30 days to appear on the leaderboard.
Assumptions	The system updates scores in real-time or scheduled intervals .

3.1.8 Certification and Badges

To encourage continuous learning, Reflect provides certificates and badges to learners upon milestone achievements. Certificates are provided on passing a final test, while badges are provided on completing specific challenges, quizzes, or high scores. All such milestone achievements are highlighted on the user profile and contribute towards leaderboard rankings.

Table 3.8: Use Case - Certification & Badges

Use Case ID	UC-08
Use Case Name	Certification and Badges
Actors	Student, System, Admin
Description	Awards certificates and badges to users for completing courses and achieving milestones.

Use Case ID	UC-08
Trigger	The user passes the Final Course Test or reaches a milestone .
Preconditions	The user must have completed all required quizzes and coding challenges.
Postconditions	A certificate is generated, and a badge is awarded to the user's profile.
Normal Flow	1. The system checks if the user has passed the final test. 2. If passed, the system generates a certificate with the user's name. 3. The system awards the user a badge and updates their profile. 4. The user can download the certificate or share it on LinkedIn.
Alternative Flows	If the user fails, they can retry the test to earn certification.
Exceptions	If certificate generation fails, the system provides a manual download link later.
Business Rules	1. Only users who pass with 70% or higher receive a certificate. 2. Badges are awarded for completing key milestones (e.g., "Completed 5 Challenges").
Assumptions	The system generates certificates in PDF format .

3.1.9 Roadmap Progress Update

As the student completes learning modules, their roadmap is updated dynamically by the system. The user can visually track their progress through a progress bar and milestone indicators. The AI examines learning patterns and can adjust the roadmap for optimal pacing.

Table 3.9: Use Case - Roadmap Progress Update

Use Case ID	UC-9
Use Case Name	Roadmap Progress Update
Actors	Student, System
Description	Updates user progress in the learning roadmap after quiz or challenge completion.
Trigger	The user completes a quiz, coding challenge, or final test .
Preconditions	The user must have an active roadmap.
Postconditions	1. The roadmap is updated with completed checkpoints.
Normal Flow	1. The user completes a quiz or challenge.

Use Case ID	UC-9
	2. The system updates their roadmap progress . 3. The system unlocks the next recommended topic . 4. The user sees an updated progress bar.
Alternative Flows	If the user resets their roadmap, progress is cleared and restarted .
Exceptions	If the system fails to update progress, it retries after some time.
Business Rules	Users must complete at least 80% of the roadmap before taking the final test.
Assumptions	The system auto-saves progress after every quiz attempt.

3.1.10 Manual Quiz and Challenge Creation

Admins are allowed to manually create quizzes and coding challenges to supplement AI-generated content. They are able to define multiple-choice questions, coding issues, difficulty levels, and test cases. This offers students high-quality, organized learning content.

Table 3.10: Use Case - Manual Quiz and Challenge Creation

Use Case ID	UC-10
Use Case Name	Manual Quiz and Challenge Creation
Actors	Admin
Description	Allows admins to manually create quizzes and coding challenges.
Trigger	The admin selects " Create Quiz " or " Add Coding Challenge ".
Preconditions	The admin must be logged in.
Postconditions	The quiz or challenge is saved and available for students.
Normal Flow	1. The admin selects Create Quiz or Add Challenge . 2. They enter questions, options, correct answers, and difficulty level . 3. The system validates and saves the new quiz/challenge. 4. Students can now attempt it.
Alternative Flows	If the admin saves as draft , the quiz remains unpublished.
Exceptions	If the system fails to save the quiz, the admin is notified.
Business Rules	Admins must provide at least 5 questions per quiz.
Assumptions	The system allows editing existing quizzes .

3.1.11 User Performance Monitoring

Administrators are able to track student performance and activity through analytics dashboards. The site provides visibility into quiz scores, coding challenge attempts, and leaderboard rankings so that instructors can assist poorly performing students.

Table 3.11: Use Case - User Performance Monitoring

Use Case ID	UC-11
Use Case Name	User Performance Monitoring
Actors	Admin, System
Description	Allows admins to track student performance based on quizzes, coding challenges, and final test results.
Trigger	The admin accesses the User Performance Dashboard .
Preconditions	The system must have stored quiz and challenge records.
Postconditions	The admin sees a detailed performance report .
Normal Flow	1. The admin selects " View Student Progress ". 2. The system retrieves quiz scores, challenge results, and leaderboard ranking . 3. The admin can filter by user, topic, or date range .
Alternative Flows	If the admin applies filters, the results update accordingly.
Exceptions	If no data is available, the system shows " No progress recorded ".
Business Rules	Only admins can access performance data.
Assumptions	The system updates performance data in real-time .

3.1.12 Content Management

Administrators can also upload and organize learning content, including text documents, PDFs, and video. This offers students quality learning content in addition to AI-generated content.

Table 3.12: Use Case - Content Management

Use Case ID	UC-12
Use Case Name	Content Management
Actors	Admin
Description	Allows admins to upload, edit, and manage learning materials for students.

Use Case ID	UC-12
Trigger	The admin selects " Manage Content " from the dashboard.
Preconditions	The admin must be logged in.
Postconditions	The uploaded or edited content is saved and accessible to students.
Normal Flow	1. The admin selects Upload New Material or Edit Existing Content. 2. They enter content details (title, description, file/video link). 3. The system validates and saves the content. 4. The content is made available to students.
Alternative Flows	If an admin saves a draft, the content remains unpublished.
Exceptions	If the upload fails, the admin is notified to retry.
Business Rules	Only admins can upload or edit content.
Assumptions	The system supports various file types (PDF, video, etc.).

3.1.13 Leaderboard and Achievement Management

Admins oversee the leaderboard and achievements system to ensure a ranking that is fair and free of abuse. They can change scoring algorithms, reset rankings, or audit cheating activity.

Table 3.13: Use Case - Leaderboard and Achievement Management

Use Case ID	UC-13
Use Case Name	Leaderboard and Achievement Management
Actors	Admin, System
Description	Allows admins to manage leaderboard rankings and student achievements.
Trigger	The admin accesses the Leaderboard Management Panel .
Preconditions	Users must have completed at least one quiz or challenge.
Postconditions	Leaderboard rankings and achievements are updated.
Normal Flow	1. The admin selects View Leaderboard or Manage Achievements. 2. The system retrieves ranking and achievement data. 3. The admin can adjust badges, points, or rankings if needed. 4. The updated leaderboard is displayed to students.
Alternative Flows	If an admin manually updates points, the system logs the change.

Use Case ID	UC-13
Exceptions	If leaderboard data fails to load, the system shows an error message.
Business Rules	Only users who have attempted at least one challenge in the last 30 days appear on the leaderboard.
Assumptions	The system updates rankings in real-time or scheduled intervals.

3.1.14 AI-Based Quiz Generation

Automatic quiz generation system utilizing AI accepts uploaded text files or YouTube videos and extracts key concepts. The concepts are turned into multiple-choice questions for testing.

Table 3.14: Use Case - AI-Based Quiz Generation

Use Case ID	UC-14
Use Case Name	AI-Based Quiz Generation
Actors	System
Description	The AI automatically generates quiz questions based on user progress.
Trigger	A student selects " Generate Quiz " for a topic.
Preconditions	The user must have an active roadmap.
Postconditions	The system provides AI-generated quiz questions.
Normal Flow	1. The user selects a topic and clicks Generate Quiz. 2. The system analyzes previous quiz attempts and topic progress. 3. The AI generates relevant questions with answer choices. 4. The user begins the quiz.
Alternative Flows	If AI fails, the system loads pre-existing quiz questions.
Exceptions	If the server is down, the user sees an error message.
Business Rules	AI-generated quizzes must align with the user's skill level.
Assumptions	The system has enough quiz data for AI training.

3.1.15 AI-Based Learning Recommendations

Reflect's AI analyzes the user's performance and recommends additional study material, quizzes, or coding challenges to strengthen weak areas.

Table 3.15: Use Case - AI-Based Learning Recommendations

Use Case ID	UC-15
Use Case Name	AI-Based Learning Recommendations
Actors	System
Description	AI recommends learning materials based on user weaknesses.
Trigger	The user completes a quiz or coding challenge.
Preconditions	The user must have attempted at least one quiz or challenge.
Postconditions	The system provides personalized learning suggestions.
Normal Flow	1. The user submits a quiz or coding challenge. 2. The AI analyzes mistakes and weak areas. 3. The system suggests articles, videos, or new quizzes .
Alternative Flows	If AI fails, the system suggests generic study materials.
Exceptions	If there's no sufficient learning data, the system informs the user.
Business Rules	Recommendations must align with the user's progress.
Assumptions	The AI system continuously updates based on user behavior.

3.1.16 Code Execution and Evaluation

The system runs coding submissions via Judge0 API, with instant feedback and test case evaluation.

Table 3.16: Use Case - Code Execution and Evaluation

Use Case ID	UC-16
Use Case Name	Code Execution and Evaluation
Actors	System
Description	Executes user-submitted code and returns test results.
Trigger	The student submits code for evaluation.
Preconditions	1. The user must be logged in. 2. The system must be connected to Judge0 API.
Postconditions	The code is executed, and feedback is provided.
Normal Flow	1. The user submits code for a challenge.

Use Case ID	UC-16
	3. The API executes the code and returns test case results. 4. The system displays output, errors, and efficiency stats.
Alternative Flows	If API is slow, the system queues execution requests.
Exceptions	If Judge0 API is down, the user is notified.
Business Rules	Users must pass at least 80% of test cases to complete a challenge.
Assumptions	The system ensures API limits are not exceeded.

3.1.17 AI-Generated Final Test Questions

AI dynamically generates final test questions based on the user's learning history. This ensures a customized difficulty level for final assessments

Table 3.17: Use Case - AI-Generated Final Test Questions

Use Case ID	UC-17
Use Case Name	AI-Generated Final Test Questions
Actors	System
Description	AI generates final test questions based on a user's learning history.
Trigger	The student begins the Final Course Test .
Preconditions	The user must have completed at least 80% of quizzes and challenges .
Postconditions	1. The system generates and presents unique test questions.
Normal Flow	1. The user starts the final test. 2. The AI selects relevant MCQs and coding challenges. 3. The system presents the final test.
Alternative Flows	If AI fails, the system loads pre-saved test questions .
Exceptions	If test generation fails, the user is given an error message.
Business Rules	Each test must be unique per user attempt.
Assumptions	The system ensures fairness in test generation.

3.2 Functional Requirements

This section describes the functional requirements of Reflect, ensuring that all features and interactions are clearly defined. Each requirement is labeled with a unique identifier, along with a description, source, rationale, business rules, dependencies, and priority.

3.2.1 User Registration and Login

The users must register for the system initially before logging in. The process of registering includes providing an email, password, and username. The credentials are checked by the system, and the user information is securely saved. The users log in with the credentials after registering. JWT tokens are used for the authentication process, and after multiple failed login attempts, the account will become locked for some time. The whole process details are provided in Table.

Table 3.18: Functional Requirement - User Registration and Login

Identifier	FR-01
Title	User Registration and Login
Requirement	The system shall allow users to register using email and password. The system shall authenticate users upon login and manage sessions securely using JWT.
Source	System Requirement
Rationale	Ensures users can securely access the system and protect their data.
Business Rule	Users must verify their email before accessing full features.
Dependencies	None
Priority	High

3.2.2 Profile Management

Profile information, including name, e-mail, and password, may be edited by users. Learning preferences, which impact the AI-driven roadmap, may also be edited by students. Profile management privileges are given to the admins but not learning preferences.

Table 3.19: Functional Requirement - Profile Management

Identifier	FR-02
Title	Profile Management
Requirement	Users shall be able to update their profile information, including name, email, password, and learning preferences.
Source	User Requirement
Rationale	Allows users to modify personal details and tailor learning experiences.
Business Rule	Email changes require re-verification.
Dependencies	FR-01 (User Registration & Login)
Priority	Medium

3.2.3 Personalized Learning Roadmap

After the user has finished the initial assessment, the system generates a personalized learning roadmap. The roadmap consists of sequential lessons, quizzes, and programming exercises, which are dynamic and change according to the user performance. The full description of this feature has been given in the table.

Table 3.20: Functional Requirement - Personalized Learning Roadmap

Identifier	FR-03
Title	Personalized Learning Roadmap
Requirement	The system shall generate a customized learning roadmap for each user based on their skill assessment.
Source	AI Module
Rationale	Provides users with a structured learning plan to ensure personalized learning.
Dependencies	FR-02 (Profile Management)

Identifier	FR-03
Priority	High

3.2.4 Quiz Attempt and Feedback

Students may take AI-generate or hand-crafted quizzes. Both quizzes contain multiple-choice questions, and the system provides immediate feedback to the answers given by the users. Correct answers, explanations, and areas where the user may need to improve are provided by the system. The entire quiz function breakdown has been explained in Table.

Table 3.21: Functional Requirement - Quiz Attempt and Feedback

Identifier	FR-04
Title	Quiz Attempt and Feedback
Requirement	Users shall be able to attempt quizzes and receive automated feedback, including score, correct answers, and explanations.
Source	User Requirement
Rationale	Provides interactive learning and ensures knowledge retention.
Dependencies	FR-03 (Personalized Roadmap)
Priority	High

3.2.5 Coding Challenge Submission and Execution

Students can solve coding challenges within the integrated code editor. They are executed through Judge0 API, providing them with immediate feedback regarding the correctness, execution time, and suggestions for optimization of the code. The whole process of executing the coding challenges is explained in table.

Table 3.22: Functional Requirement - Coding Challenge Submission and Execution

Identifier	FR-05
Title	Coding Challenge Submission and Execution
Requirement	Users shall be able to submit coding challenges, execute code using the Judge0 API, and receive feedback in real-time.
Source	System Requirement

Identifier	FR-05
Rationale	Provides hands-on coding practice with automated evaluation.
Dependencies	None
Priority	High

3.2.6 Progress Tracking

The system provides visual indications for tracking user progress. When user complete lessons, quizzes, and challenges, the roadmap is updated in real-time, and users can see achievements and areas for improvement. The full details for tracking progress are summarized in table.

Table 3.23: Functional Requirement - Progress Tracking

Identifier	FR-06
Title	Progress Tracking
Requirement	The system shall track user progress and update completion indicators for lessons, quizzes, and coding challenges.
Source	User Requirement
Rationale	Allows users to monitor their progress and stay motivated.
Dependencies	FR-03 (Personalized Roadmap)
Priority	Medium

3.2.7 AI-Based Learning Recommendations

The AI monitors the performance of the users and recommends additional study materials, quizzes, or coding exercises to strengthen weak areas.

Table 3.24: Functional Requirement - AI-Based Learning Recommendations

Identifier	FR-07
Title	AI-Based Learning Recommendations
Requirement	The system shall recommend additional study materials and challenges based on user performance.
Source	AI System
Rationale	Ensures users receive targeted learning materials to improve weak areas.
Dependencies	FR-06 (Progress Tracking)
Priority	High

3.2.8 Code Execution and API Integration

The Judge0 API allows for code execution in real-time, testing code against multiple test cases and returning the result immediately. More details regarding the manner in which this function works is provided in table.

Table 3.25: Functional Requirement - Code Execution and API Integration

Identifier	FR-08
Title	Code Execution & API Integration
Requirement	The system shall execute coding submissions in real time using Judge0 API and return results instantly.
Source	System Requirement
Rationale	Enables real-time coding assessment for users.
Dependencies	FR-05 (Coding Challenge Submission)
Priority	High

3.3 Non-Functional Requirements

This chapter details the Non-Functional Requirements (NFRs) for Reflect so that the system meets usability, performance, reliability, security, maintainability, and compatibility requirements. Whereas functional requirements dictate what the system should do, non-functional requirements dictate how the system should perform under given conditions. Non-functional requirements are quantifiable, testable, and essential to enable a smooth user experience while maintaining scalability, security, and efficiency.

Subsections Covered in This Section:

- **Usability Requirements** – Defines user interaction and accessibility expectations.
- **Performance Requirements** – Specifies system response time and efficiency.
- **Reliability and Availability Requirements** – Ensures system stability and fault tolerance.
- **Security Requirements** – Outlines data protection and authentication measures.
- **Maintainability and Scalability Requirements** – Defines system flexibility for updates and growth.
- **Compatibility Requirements** – Ensures cross-platform accessibility and third-party integration.

3.3.1 Usability Requirements

To ensure a seamless and user-friendly experience, Reflect is designed with the following usability requirements:

- The system shall have an intuitive user interface (UI), allowing users to navigate all features without prior training.
- First-time users shall be able to complete registration and begin learning within two minutes to ensure easy onboarding.
- The platform shall provide tooltips and interactive guides to assist users in navigating quizzes, coding challenges, and study roadmaps.
- The system shall support keyboard shortcuts for quick navigation and quiz selection to enhance usability.
- The UI shall be responsive and mobile-friendly, ensuring optimal performance on desktops, tablets, and future mobile support.

- Users shall be able to resume unfinished quizzes or coding challenges in case of a logout or system crash, preventing progress loss.

3.3.2 Performance Requirements

Reflect must be able to handle multiple users efficiently while maintaining fast response times:

- The system shall generate AI-based quizzes within three seconds using the Groq API.
- 95% of webpages shall load within three seconds over a 10 Mbps internet connection.
- The system shall support at least 10,000 concurrent users without experiencing performance degradation.
- The leaderboard shall update rankings every 10 minutes, ensuring real-time progress tracking.
- The platform shall maintain at least 99.5% uptime, minimizing downtime and ensuring high availability.

3.3.3 Reliability and Availability Requirements

To ensure system stability and prevent data loss, the following reliability measures will be implemented:

- The system shall automatically save user progress every 30 seconds, preventing accidental loss of data.
- The platform shall provide graceful error handling, displaying user-friendly messages instead of technical errors when failures occur.
- The system shall retry failed API requests up to three times before notifying the user of an issue.
- The system shall perform daily automated data backups every 24 hours to prevent data loss.
- In the event of a server failure, the system shall restore services within 30 minutes, ensuring minimal downtime.

3.3.4 Security Requirements

Security is a top priority for Reflect to ensure data protection and secure access:

- The system shall use JWT authentication to provide secure login sessions and session management.
- All user passwords shall be hashed and salted using industry-standard encryption techniques such as bcrypt.
- Role-based access control (RBAC) shall be implemented to ensure admin functionalities remain restricted to authorized personnel.
- Sensitive user data, including quiz results and coding history, shall be stored in an encrypted format to prevent unauthorized access.
- The system shall log all failed login attempts and lock an account after five consecutive failed login attempts, mitigating brute-force attacks.
- Users shall be automatically logged out after 15 minutes of inactivity to prevent unauthorized access.

3.3.5 Maintainability and Scalability Requirements

To ensure long-term system growth and maintainability, the following requirements will be enforced:

- The system shall be designed using a modular architecture, allowing independent updates to the frontend, backend, and AI services.
- The platform shall scale up to 10,000 users without a decline in performance.
- The database schema shall be optimized for fast retrieval, ensuring that queries return results within one second even under high traffic conditions.
- The system shall undergo unit, integration, and performance testing before deployment to ensure stability and reliability.
- The backend API shall follow RESTful design principles, allowing seamless future expansion and third-party integrations.

3.3.6 Compatibility Requirements

Reflect is designed to be cross-platform compatible, ensuring accessibility across different devices and environments:

- The system shall be compatible with the latest versions of major web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.
- The platform shall be mobile-responsive, ensuring that all features function optimally on screens as small as 5 inches, supporting future mobile accessibility.
- The Reflect API shall be accessible via RESTful endpoints, allowing third-party applications to integrate with the system.

Chapter 4

Design and Architecture

4. Design and Architecture

4.1 System Architecture

The architecture of "Reflect" is built upon the robust **Three-Tier Web Architecture** model. This structural design was chosen to ensure separation of concerns, scalability, and maintainability. The system is divided into three logical layers, each operating independently but communicating through secure API interfaces.

1. Presentation Layer (Frontend)

The Presentation Layer is the client-side component responsible for user interaction. It is developed using **Next.js (React)**. This layer runs in the user's web browser and is responsible for:

- **Rendering the UI:** Displaying the dashboard, roadmaps, and quiz interfaces.
- **State Management:** Handling the user's session state (logged in/out) and temporary data (current quiz score) using React Context and Redux.
- **API Communication:** Making asynchronous HTTP requests (GET/POST) to the backend server using Axios.
- **Responsiveness:** Ensuring the application adapts to various screen sizes (Desktop, Tablet, Mobile) using Tailwind CSS.

2. Application Layer (Backend)

The Application Layer serves as the "brain" of the system. It is hosted on a Node.js server using the **Express.js** framework. This layer contains the core business logic and acts as an orchestrator between the user and the data/external services. Its responsibilities include:

- **Authentication:** Verifying JWT tokens and protecting private routes.
- **Prompt Engineering:** Constructing the specific text prompts sent to the Groq AI API to ensure high-quality responses.
- **Proxying:** Acting as a secure middleman for the Judge0 API to hide API keys from the client.
- **Data Validation:** Sanitizing all user inputs to prevent SQL Injection or NoSQL Injection attacks.

3. Data Layer (Database)

The Data Layer is the persistent storage system. We utilize **MongoDB (Atlas)**, a NoSQL database. MongoDB was selected because the data structure of a "Learning Roadmap" is hierarchical and nested (Trees of topics and sub-topics), which fits perfectly into a JSON-like

Document model.

- **User Store:** Stores credentials and profile data.
- **Content Store:** Stores the generated roadmaps and quiz history.
- **Session Store:** Stores refresh tokens for long-running sessions.

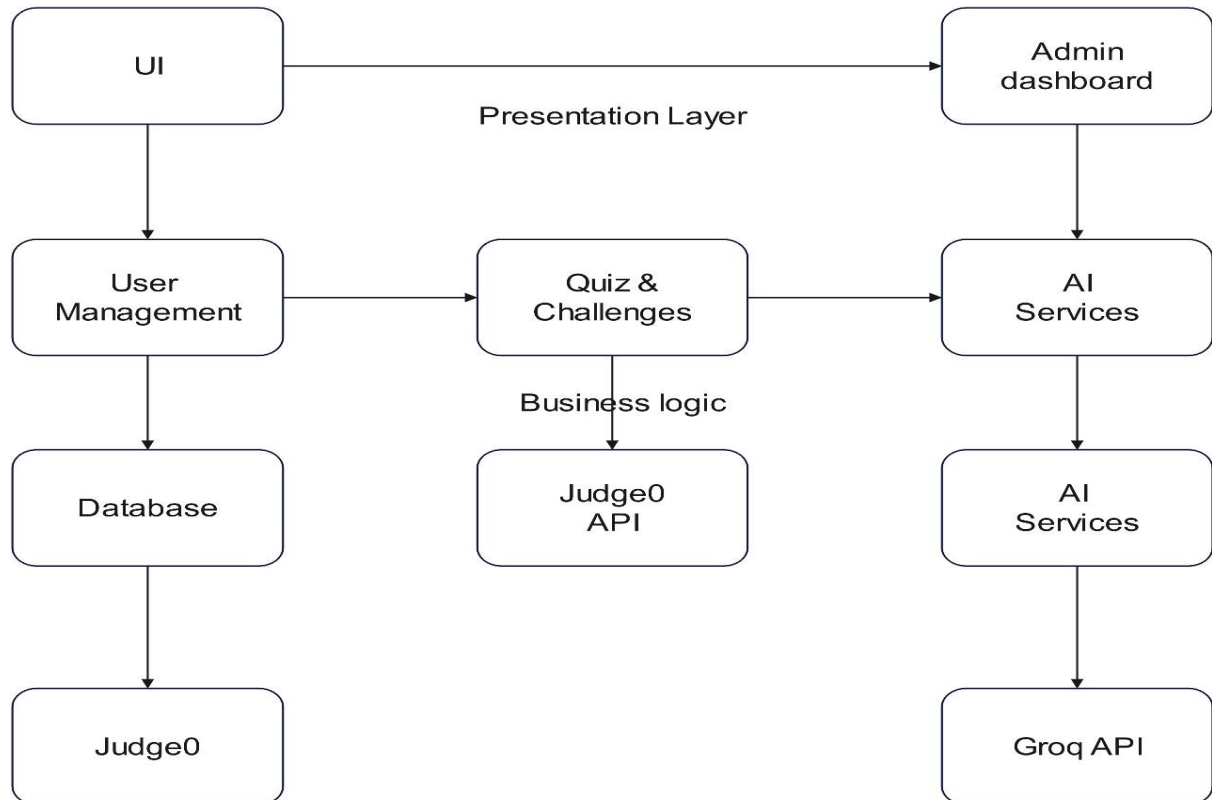


Figure 4.1 : Three-tier architecture Diagram

4.2 Data Representation

The Data Representation section outlines the schema design used within MongoDB. Unlike SQL databases, we do not use rigid tables, but rather flexible "Collections." However, strict schema validation is enforced at the application level using Mongoose.

Table 4.1: Data Representation

Attribute	Data Type	Description
User Collection		Primary storage for account info
<code>_id</code>	ObjectId	Unique primary key generated by MongoDB.
<code>username</code>	String	The unique display name of the student.

Attribute	Data Type	Description
email	String	Validated email address used for login.
password	String	Bcrypt-hashed string (never plain text).
role	String	Enum: ["student", "admin"].
createdAt	Date	Timestamp of registration.
Roadmap Collection		Storage for AI-generated plans
_id	ObjectId	Unique identifier for the roadmap.
user_id	ObjectId	Foreign key linking to User Collection.
title	String	E.g., "Introduction to Python".
nodes	Array<Object>	List containing Topic Name, Description, and Status.
progress	Integer	Percentage (0-100) of completion.
Quiz Collection		Storage for assessment history
_id	ObjectId	Unique identifier for the quiz attempt.
roadmap_id	ObjectId	Link to the specific roadmap module.
questions	Array<Object>	The list of questions generated by AI.
score	Integer	The final score achieved.
Submission Collection		Storage for code execution logs

Attribute	Data Type	Description
_id	ObjectId	Unique identifier for the code run.
code	String	The raw source code written by the user.
language_id	Integer	ID mapping to Judge0 (e.g., 71 for Python).
output	String	The stdout returned by the compiler.
status	String	"Accepted", "Compilation Error", or "Runtime Error".

4.3 Process Flow / Representation

The Process Flow diagram illustrates the journey of a user through the system. It demonstrates the cyclic nature of the "Reflect" learning model: **Assess -> Learn -> Practice -> Re-assess.**

1. **Registration:** User enters the system.
2. **Assessment:** User requests a roadmap for a specific skill.
3. **Generation:** System calls Groq API to build the roadmap.
4. **Learning:** User selects a node on the roadmap.
5. **Testing:** User takes a quiz or submits code.
6. **Feedback:** System updates the roadmap progress based on the result.

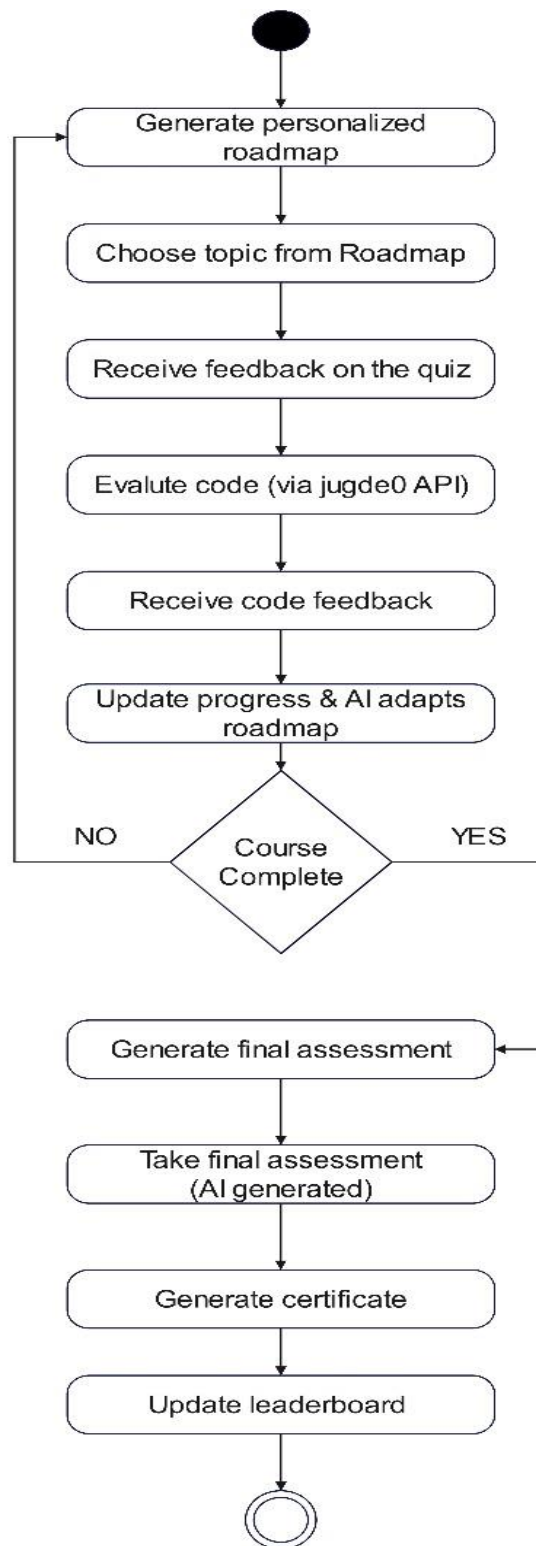


Figure 4.2 : Process Flow Representation

4.4 Class Diagram

Relationships:

- Inheritance: Student and Admin inherit from User.
- Associations: Student owns Roadmaps, Quizzes, Challenges, and Certificates.
- Aggregation: Roadmap contains multiple topics; Quiz contains multiple questions.

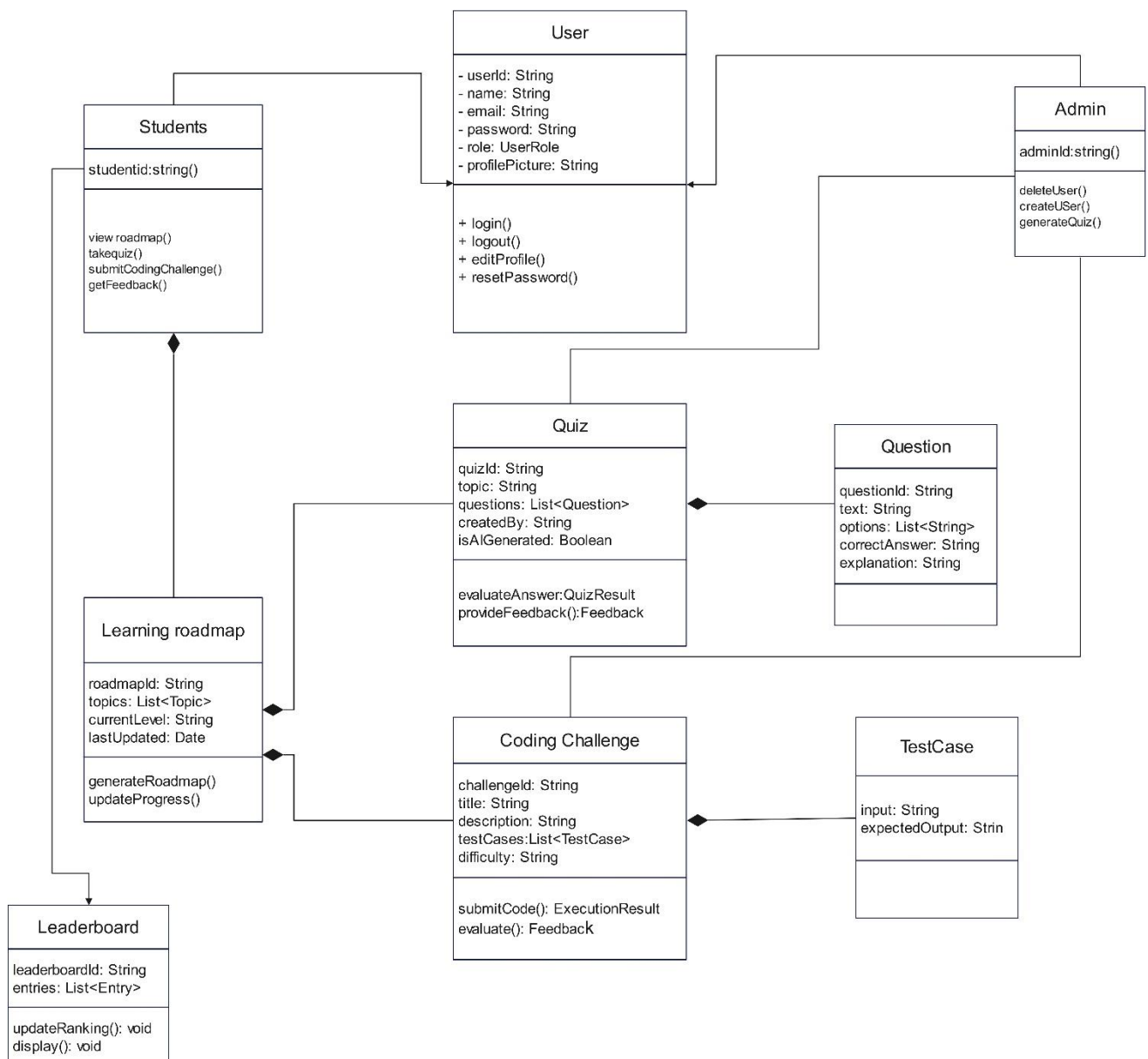


Figure 4.3: Primary Class Diagram

4.5 Sequence Diagram

Actors

- **Student:** The main user interacting with the platform.
- **Reflect System:** The core application handling user requests, quiz generation, and feedback.
- **Groq API:** An external API responsible for generating quizzes using AI.
- **Judge0 API:** An external API used for real-time code execution and evaluation.
- **AI-Powered Roadmap Generation:** To visually demonstrate the real-time interactions between the Student, Reflect System, Groq API, and Judge0 API.

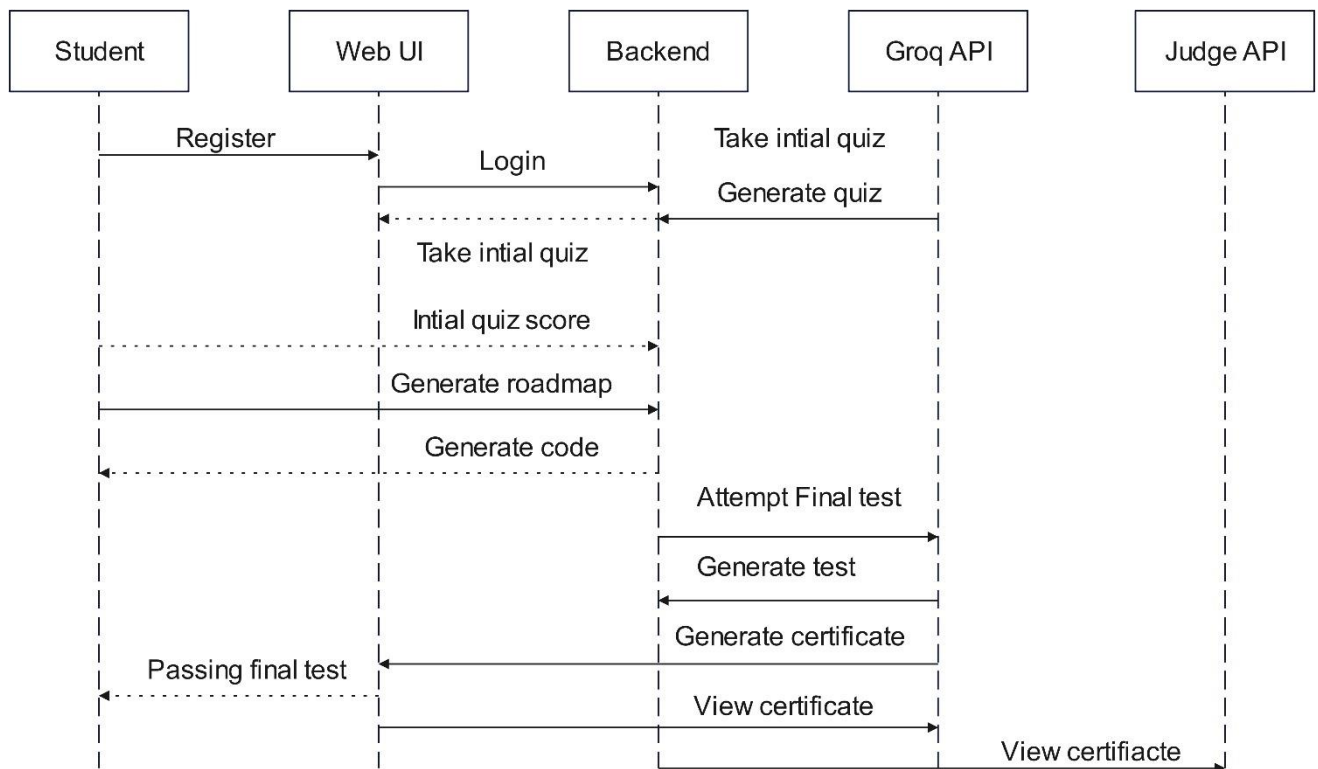


Figure 4.4 Sequence Diagram

Chapter 5

Implementation

5. Implementation

5.1 Frontend Development

The frontend of "Reflect" was crafted to prioritize user experience (UX) and performance. We utilized **Next.js 14**, a React framework that supports Server-Side Rendering (SSR). SSR is crucial for our application because it allows the roadmap data to be pre-rendered on the server before being sent to the client, resulting in a significantly faster initial load time compared to standard React apps.

Key Technologies & Libraries:

- **React Hooks:** We used `useState` for handling form inputs and `useEffect` for fetching data on component mount.
- **Tailwind CSS:** A utility-first framework that allowed us to build a completely custom, responsive design without fighting against default browser styles. We implemented a "Dark Mode" theme using Tailwind's dark variant.
- **Monaco Editor:** This is the exact same code editor engine used in VS Code. By integrating `@monaco-editor/react`, we provided users with a familiar coding environment that supports syntax highlighting, error linting, and auto-completion for Python and JavaScript.
- **Axios:** Used for efficient HTTP requests with built-in interceptors to automatically attach the Authorization header to every request.

5.2 Backend Development

The backend logic is housed in a **Node.js** runtime environment using **Express.js**. The backend is structured using the **MVC (Model-View-Controller)** pattern to ensure code organization.

Key Components:

- **Middleware:** We wrote custom middleware functions for `verifyToken` (security) and `errorHandler` (to catch API failures gracefully).
- **Controllers:**
 - `AuthController.js`: Handles login, registration, and hashing.
 - `AIController.js`: Manages the connection to Groq. This controller includes a "Retry Strategy" if the AI returns malformed data, the controller automatically retries the request up to 3 times.
 - `CompilerController.js`: Manages the connection to Judge0.

- **Mongoose Models:** Defined strict schemas for data validation (e.g., ensuring a user cannot register without a valid email format).

5.3 Integration of AI Algorithms

The integration of Artificial Intelligence is the differentiator for "Reflect."

1. Groq API (Llama 3): We selected Groq for its inference speed. The latency is sub second, which is vital for a real-time user experience.

- **Prompt Engineering:** We spent significant time refining the "System Prompt."
 - *Prompt:* "You are a strict JSON generator for a coding education platform. You must output a JSON array of objects. Do not include any conversational text."
 - *Outcome:* This ensures the backend can programmatically parse the AI's output without crashing.

2. Judge0 API: We utilized Judge0 for secure remote code execution.

- **Security:** Running user code is dangerous. Judge0 handles this by spinning up isolated Docker containers for every submission. If a user writes an infinite loop (`while(true)`), the container is killed after 5 seconds, protecting our server from crashing.

5.4 User Interface

The User Interface (UI) of "Reflect" was designed with a primary focus on cognitive ease and minimalism. In an educational platform, it is critical that the interface does not compete with the learning material for the user's attention. Therefore, we adopted a "distraction-free" design philosophy, utilizing ample whitespace, clear typography, and a cohesive color hierarchy to guide the user's eye to the most important actions whether that is taking a quiz or writing code.

1. Sign Up:

The Signup Page is designed to lower the barrier to entry for new students. It mirrors the split-screen layout of the Login page for consistency. The form collects essential information Full Name, Email, and Password while implementing real-time client-side validation. As the user types, the interface provides immediate visual cues (such as red borders for invalid emails) to prevent submission errors. A "Join for free" call-to-action is emphasized, and users with existing accounts are provided a clear link to redirect to the Login screen.

Fig 5.1 Signup

2. Login:

The Login Page serves as the secure entry point for existing users. It utilizes a split-screen layout to enhance user experience. The left panel reinforces the platform's value proposition, highlighting key features such as "Adaptive Roadmaps," "Real-Time Feedback," and "Measurable Growth" to remind users of the system's benefits. The right panel contains a clean, distraction-free form for authentication, requiring an email address and password. It includes essential links for "Forgot password?" and new user registration, ensuring a smooth navigation flow.

Fig 5.2 Login

3. User Dashboard:

The Dashboard acts as the central hub for the student's learning journey. The header features a personalized greeting ("Good evening, Khuram") and visualizes immediate progress with a "Weekly Goal" bar, a "Day Streak" counter, and "XP" (Experience Points) to gamify the experience. The main content area is divided into two key sections:

- **Quizzes:** Displays a card view of recent assessments (e.g., OOP, Python, React JS) with their completion status, difficulty tags (Easy/Hard), and scores.
- **Roadmaps:** Provides quick access to active learning paths, such as "Get A Job In Tech In Mobile App Development," showing the number of steps and the expected completion date.

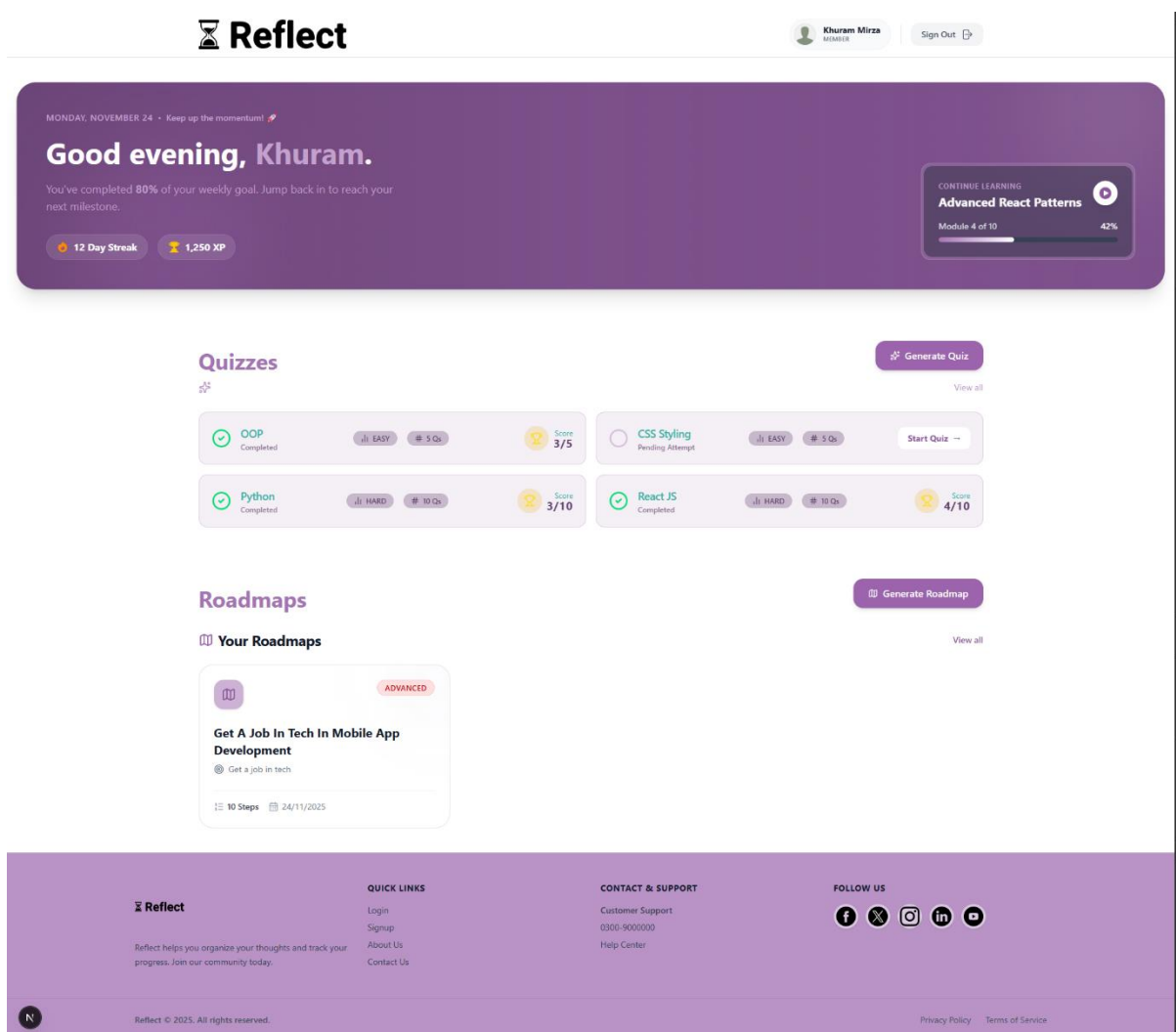


Fig 5.3 User Dashboard

4.Roadmap Generation Wizard:

This screen illustrates the interactive onboarding process used to gather user requirements for AI generation. It uses a multi-step wizard format (currently showing Step 3 of 10). The interface asks specific questions, such as "Which field are you most interested in?", presenting options like Web Development, Data Science, and Cybersecurity. This data is passed to the Groq API to ensure the generated roadmap is highly tailored to the user's specific career goals.

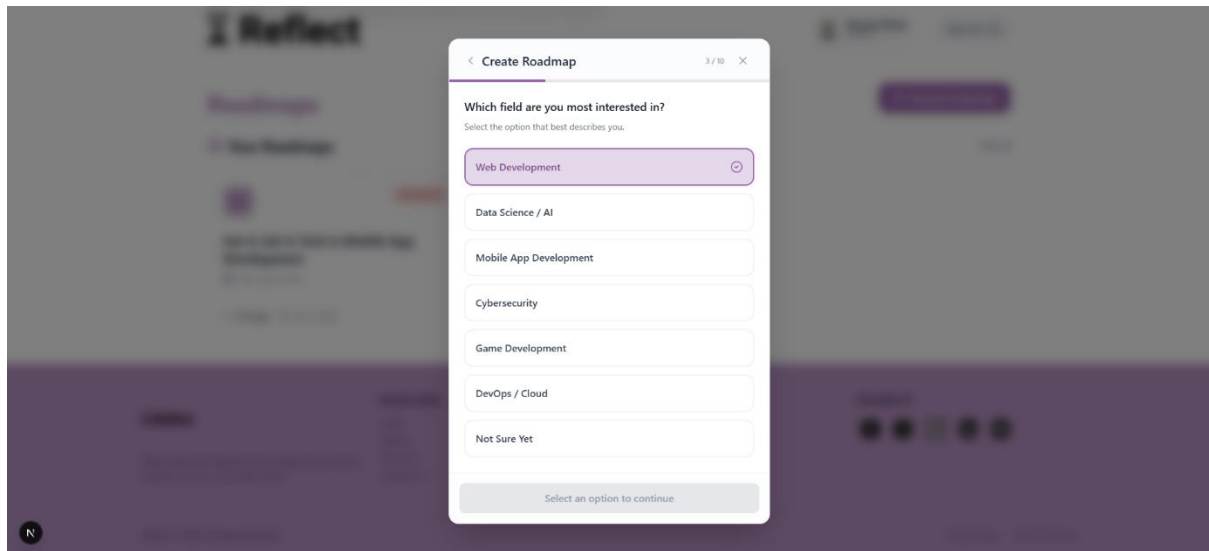


Fig 5.4 Roadmap Generation

5.Roadmap Details View:

Once a roadmap is generated, this timeline view presents the structured curriculum. The example shown, "Mobile App Development," breaks the course down into sequential modules (e.g., "1. Kotlin Fundamentals," "2. React Native Basics"). Each module card provides a brief summary of the learning objectives, the estimated time to complete (e.g., ~2 hours), and the key concepts covered. The vertical timeline connector visually guides the user from beginner topics to advanced skills.

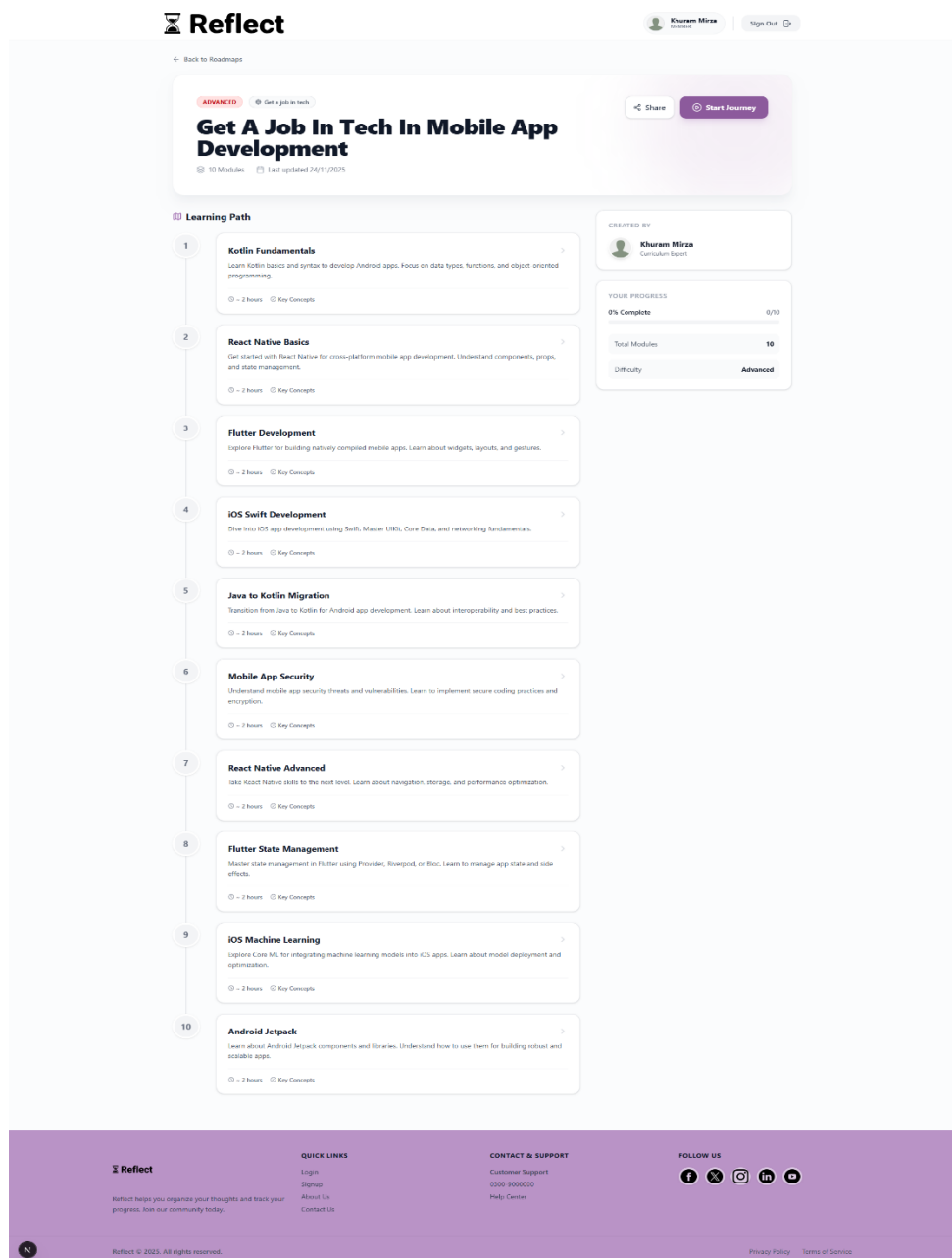


Fig 5.5 Roadmap Details

6.Module Drill-Down:

This interface appears when a user selects a specific module from the main roadmap (e.g., "iOS Swift Development"). It provides a granular breakdown of that specific topic. The page lists detailed sub-topics or "Checkpoints" such as "Introduction to Swift," "Xcode Environment," "Control Flow," and "Error Handling." Each sub-topic allows the user to view specific resources and mark the item as complete, offering fine-grained progress tracking within a broader subject.

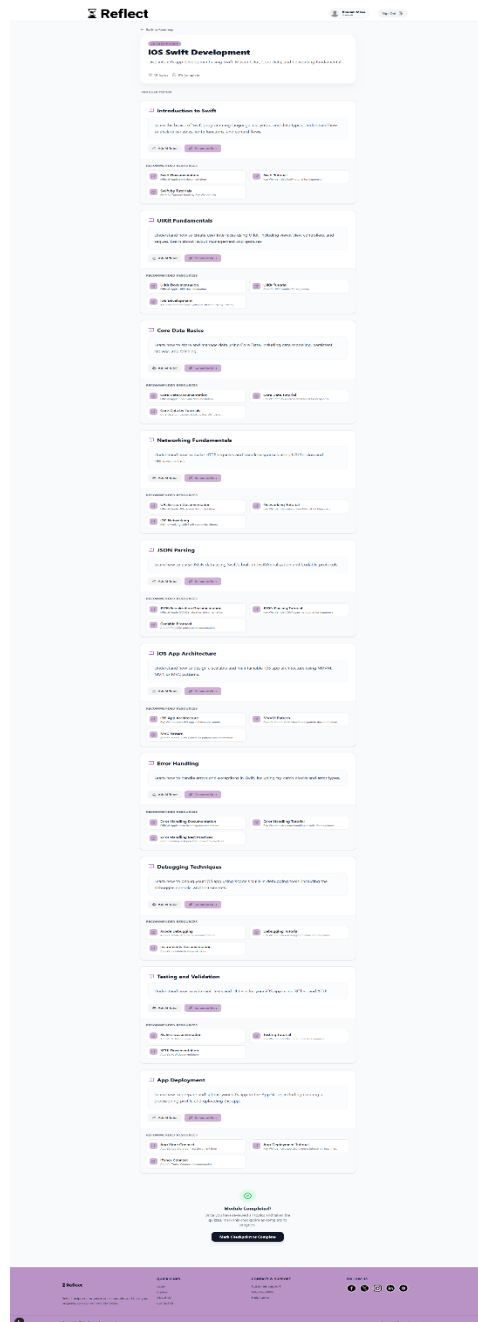


Fig 5.6 Modules

7. Quiz Configuration:

This modal interface allows users to request a custom assessment from the AI. It provides fields for the user to define the scope of the quiz, including the Topic (e.g., "Machine Learning") and a Description context (e.g., "How to train Models"). Dropdown menus allow for the selection of Difficulty (Expert) and the Number of Questions (10). This configuration ensures the AI generates relevant questions that match the user's current study needs.

The image shows a web interface for 'Reflect AI' with a quiz configuration modal open. The modal is titled 'Machine Learning' and contains ten multiple-choice questions. The questions cover topics such as generative vs. discriminative models, regularization, ensemble methods, neural network activation functions, batch normalization, overfitting, supervised vs. unsupervised learning, clustering algorithms, transfer learning, and evaluation metrics. The interface includes a 'Submit Quiz' button at the bottom of the modal. The footer of the page contains contact information, quick links, and social media icons.

Reflect AI

Machine Learning
Answer all questions to complete the quiz.

1. What is the primary difference between a generative model and a discriminative model in machine learning?

☐ Generative models are used for classification, while discriminative models are used for regression.

☐ Generative models are used for regression, while discriminative models are used for classification.

☐ Generative models learn to model the distribution of data, while discriminative models learn to predict a target variable.

☐ Generative models learn to predict a target variable, while discriminative models learn to model the distribution of data.

2. What is the purpose of regularization in machine learning models?

☐ To increase the complexity of the model.

☐ To decrease the complexity of the model and prevent overfitting.

☐ To increase the training time of the model.

☐ To decrease the training time of the model.

3. Which of the following machine learning algorithms is an example of an ensemble method?

☐ Decision Trees

☐ Random Forest

☐ Support Vector Machines

☐ K-Nearest Neighbors

4. What is the difference between a neural network's activation function and its loss function?

☐ The activation function is used to calculate the loss, while the loss function is used to update the model's weights.

☐ The activation function is used to calculate the loss, while the loss function is used to calculate the error.

☐ The activation function introduces non-linearity into the model, while the loss function calculates the difference between predicted and actual values.

☐ The activation function calculates the difference between predictions and actual values, while the loss function introduces non-linearity into the model.

5. What is the purpose of batch normalization in deep neural networks?

☐ To normalize the input data.

☐ To normalize the output data.

☐ To normalize the activations of each layer.

☐ To normalize the weights of each layer.

6. Which of the following is a common problem in machine learning models, where the model becomes too complex and starts to fit the noise in the training data?

☐ Underfitting

☐ Overfitting

☐ Bias-variance tradeoff

☐ Regularization

7. What is the difference between a supervised and an unsupervised machine learning algorithm?

☐ Supervised algorithms are used for regression, while unsupervised algorithms are used for classification.

☐ Supervised algorithms are used for classification, while unsupervised algorithms are used for regression.

☐ Supervised algorithms are trained on labeled data, while unsupervised algorithms are trained on unlabeled data.

☐ Supervised algorithms are trained on unlabeled data, while unsupervised algorithms are trained on labeled data.

8. Which of the following machine learning algorithms is an example of a clustering algorithm?

☐ K-Nearest Neighbors

☐ Support Vector Machines

☐ K-Means Clustering

☐ Decision Trees

9. What is the purpose of transfer learning in deep neural networks?

☐ To train a model from scratch on a new dataset.

☐ To fine-tune a pre-trained model on a new dataset.

☐ To use a pre-trained model as a feature extractor.

☐ All of the above.

10. Which of the following is a common evaluation metric for machine learning models that measures the proportion of correctly classified instances?

☐ Accuracy

☐ Precision

☐ Recall

☐ F1 Score

Submit Quiz

Reflect AI
Contact: support@reflect.ai or 1-800-REFLECT-001
© 2023 Reflect AI. All rights reserved.

QUICK LINKS
Home
Features
Pricing
Contact Us

CONTACT & SUPPORT
Customer Support
1-800-REFLECT-001
Email: support@reflect.ai

FOLLOW US
Facebook
Twitter
LinkedIn
Instagram

Privacy Policy | Terms of Service

Fig 5.7 Quiz configuration

8.Active Quiz Attempt:

This screen represents the assessment environment where the user actively answers questions. It features a vertical list of AI-generated Multiple-Choice Questions (MCQs) related to the selected topic (e.g., Machine Learning). Each question block presents the query and four potential options. The interface is designed to be minimal to maintain user focus. A "Submit Quiz" button is located at the bottom to finalize the attempt and send answers to the backend for grading.

The screenshot displays the 'Reflect' platform's quiz attempt interface for the topic 'Machine Learning'. The interface is clean and modern, with a purple header bar containing the 'Reflect' logo and user information. The main content area is a vertical list of ten multiple-choice questions, each with a question text and four radio button options. The questions cover various machine learning concepts, including generative vs. discriminative models, regularization, ensemble methods, neural network activation functions, batch normalization, overfitting, supervised vs. unsupervised learning, clustering algorithms, transfer learning, and evaluation metrics. A 'Submit Quiz' button is located at the bottom of the question list. The footer contains links for 'About Us', 'Contact Us', 'Privacy Policy', and 'Terms of Service'.

Machine Learning
Answer all questions to complete the quiz.

1. What is the primary difference between a generative model and a discriminative model in machine learning?

☐ Generative models are used for classification, while discriminative models are used for regression.

☐ Generative models are used for regression, while discriminative models are used for classification.

☐ Generative models learn to model the distribution of data, while discriminative models learn to predict a target variable.

☐ Generative models learn to predict a target variable, while discriminative models learn to model the distribution of data.

2. What is the purpose of regularization in machine learning models?

☐ To increase the complexity of the model.

☐ To decrease the complexity of the model and prevent overfitting.

☐ To increase the training time of the model.

☐ To decrease the training time of the model.

3. Which of the following machine learning algorithms is an example of an ensemble method?

☐ Decision trees.

☐ Random forest.

☐ Support vector machines.

☐ K-Nearest Neighbors.

4. What is the difference between a neural network's activation function and its loss function?

☐ The activation function is used to calculate the loss, while the loss function is used to update the model's weights.

☐ The activation function is used to update the model's weights, while the loss function is used to calculate the loss.

☐ The activation function introduces non-linearity into the model, while the loss function calculates the difference between predictions and actual values.

☐ The activation function calculates the difference between predictions and actual values, while the loss function introduces non-linearity into the model.

5. What is the purpose of batch normalization in deep neural networks?

☐ To normalize the input data.

☐ To normalize the output data.

☐ To normalize the activations of each layer.

☐ To normalize the weights of each layer.

6. Which of the following is a common problem in machine learning models, where the model becomes too complex and starts to fit the noise in the training data?

☐ Underfitting.

☐ Overfitting.

☐ Bias-variance tradeoff.

☐ Regularization.

7. What is the difference between a supervised and an unsupervised machine learning algorithm?

☐ Supervised algorithms are used for regression, while unsupervised algorithms are used for classification.

☐ Supervised algorithms are used for classification, while unsupervised algorithms are used for regression.

☐ Supervised algorithms are trained on labeled data, while unsupervised algorithms are trained on unlabeled data.

☐ Supervised algorithms are trained on unlabeled data, while unsupervised algorithms are trained on labeled data.

8. Which of the following machine learning algorithms is an example of a clustering algorithm?

☐ K-Nearest Neighbors.

☐ Support Vector Machines.

☐ K-Means Clustering.

☐ Decision Trees.

9. What is the purpose of transfer learning in deep neural networks?

☐ To train a model from scratch on a new dataset.

☐ To fine-tune a pre-trained model on a new dataset.

☐ To use a pre-trained model as a feature extractor.

☐ All of the above.

10. Which of the following is a common evaluation metric for machine learning models that measures the proportion of correctly classified instances?

☐ Accuracy.

☐ Precision.

☐ Recall.

☐ F1 Score.

Submit Quiz

Reflect
Reflect helps you improve your thoughts and track your progress, one day at a time.

Quick Links
Home
About Us
Contact Us

Contact Us
Customer Support
011-11-11111111
info@reflect.in

Follow Us
Facebook
Twitter
Instagram
LinkedIn

Reflect © 2020. All rights reserved. Privacy Policy | Terms of Service

Fig 5.8 Quiz Attempt

9. Quiz History:

The Quiz History page serves as an archive of the user's performance. Top-level statistics cards display the "Total Quizzes" taken and "Total Questions" answered. Below, a list of past attempts allows users to review their scores across different subjects (Machine Learning, OOP, Python). This view helps users track their improvement over time and identify subjects that may require re-visiting.

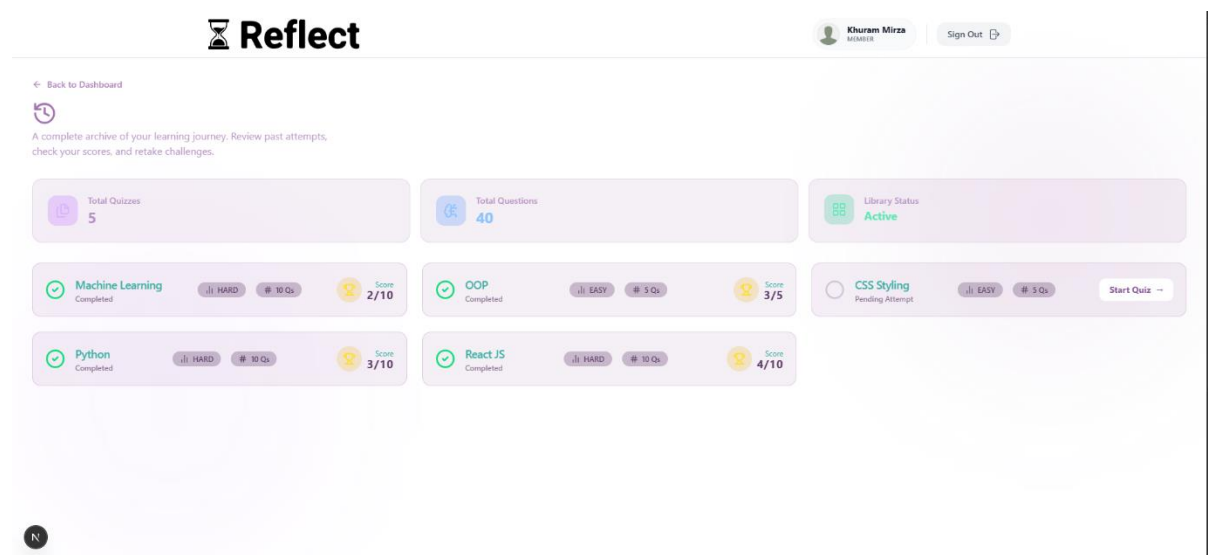


Fig 5.9 Quiz History

10. Quiz Results & Feedback:

Upon submitting a quiz, this modal provides immediate, detailed feedback.

- **Score:** The top section displays the final result (e.g., 2/10).
- **Analysis:** It lists every question, showing the "Your Answer" versus the "Correct Answer."
- **Color Coding:** Red highlights indicate incorrect answers, while green indicates correct ones. This immediate feedback loop allows users to identify their knowledge gaps instantly without needing to consult external resources.

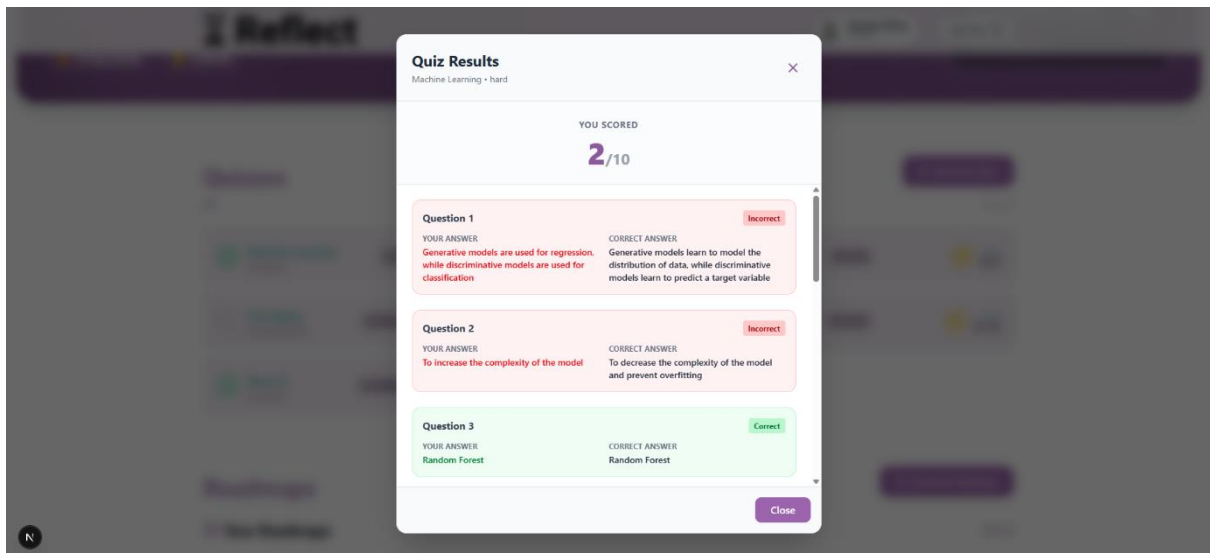


Fig 5.10 Quiz Results

11.Contact & Support Page:

The Contact page provides users with channels to seek support or provide feedback. The left panel displays static contact information, including phone numbers, email addresses, and physical office locations, establishing platform credibility. The right panel features a "Send us a message" form, allowing users to submit specific queries regarding technical issues or partnership opportunities directly to the administration team.

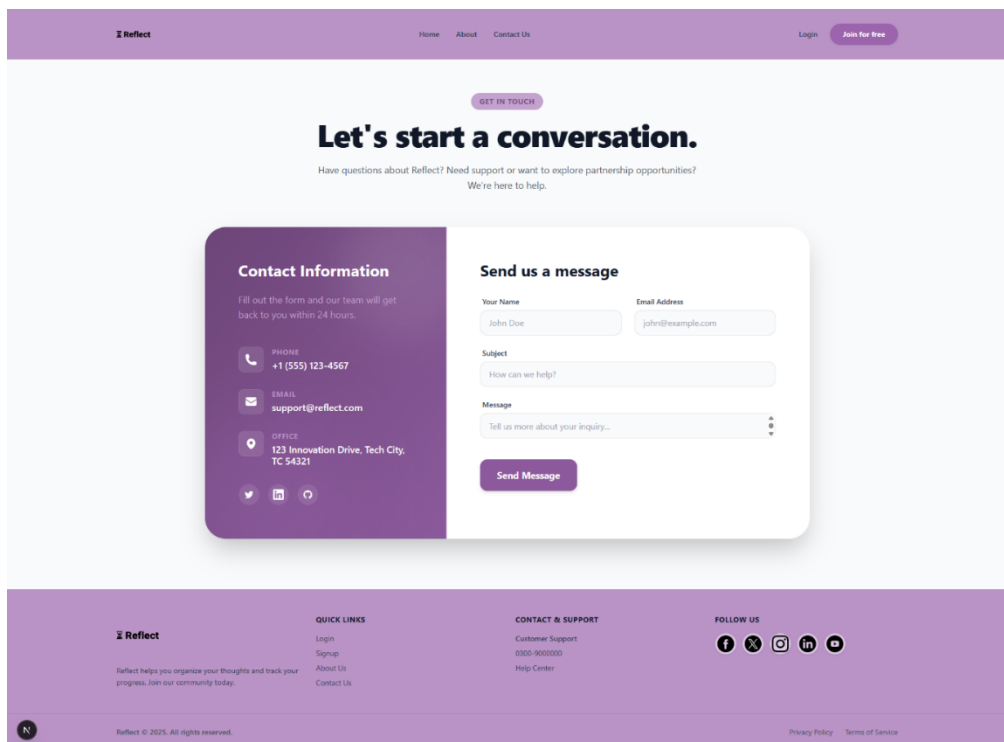


Fig 5.11 Contact and Support Page

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

6.1 Manual Testing

Manual testing was the first line of defense. We navigated through the application as a user would, attempting to break functionality by performing unexpected actions.

Table 6.1 : Manual Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
MT-01	User Registration	User enters valid details and clicks Signup. System redirects to Login.	User was redirected successfully. Database shows new entry.	Pass
MT-02	Duplicate Email	User tries to register with an existing email.	System displays “Email already in use” error.	Pass
MT-03	Generate Roadmap	User clicks “Generate”. Loader appears. Roadmap renders.	Loader appeared. Roadmap rendered in 3.5s.	Pass
MT-04	Code Execution	User runs print(“Hello”).	Output window shows “Hello”.	Pass
MT-05	Infinite Loop	User runs while(True): pass.	System returns “Time Limit Exceeded” after 5s.	Pass
MT-	Logout	User clicks Logout.	Token removed	Pass

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
06			from storage. Redirected to Home.	
MT-07	Mobile View	Access site on Mobile (375px width).	Menu collapses into Hamburger icon.	Pass

6.2 Unit Testing

Unit testing focused on ensuring individual backend functions performed correctly in isolation. We used **Jest** for testing.

Table 6.2 : Unit Testing

Test Case ID	Component / Function	Input	Expected Output	Status
UT-01	emailValidator()	test@domain (Invalid)	False	Pass
UT-02	emailValidator()	test@domain.com (Valid)	True	Pass
UT-03	passwordHasher()	secret123	String starting with \$2b\$	Pass
UT-04	promptBuilder()	Topic: "Java"	String containing "Generate JSON for Java"	Pass

Test Case ID	Component / Function	Input	Expected Output	Status
UT-05	jsonSanitizer()	String with markdown JSON	Clean JSON Object	Pass

6.3 Functional Testing

Functional testing verified that the system met the specific requirements defined in the SRS.

Table 6.3 : Functional Testing

Test Case ID	Feature Tested	Scenario	Outcome	Status
FT-01	Roadmap Generation	User requests a roadmap for "C++ Pointers".	Backend successfully calls Groq and saves a 5-step roadmap to DB.	Pass
FT-02	Quiz Scoring	User answers 3/5 questions correctly.	System calculates 60% score and updates User XP.	Pass
FT-03	Route Protection	Unauthenticated user tries to access /profile.	Middleware intercepts request and returns 401 Unauthorized.	Pass
FT-04	Profile Update	User changes display name.	Dashboard immediately reflects the new name.	Pass

6.4 Integration Testing

Integration testing ensured that the different modules (Frontend, Backend, Database, APIs) communicated correctly.

Table 6.4 : Integration Testing

Test ID	Integration Point	Description	Status
IT-01	Frontend + Backend	React components successfully fetch data from Express API.	Pass
IT-02	Backend + MongoDB	User profiles and Roadmaps persist after server restart.	Pass
IT-03	Backend + Groq API	AI generates valid JSON responses within 3 seconds.	Pass
IT-04	Backend + Judge0	Compiler receives code and returns execution logs accurately.	Pass

Chapter 7

Conclusion and Future Work

7. Conclusion and Future Work

7.1 Conclusion

The development of "**Reflect – Personalized AI Learning Assistant**" has successfully demonstrated the immense potential of integrating Generative AI into educational technology. By moving away from the static, linear models of the past, Reflect offers a glimpse into the future of personalized learning.

The system successfully met all primary objectives:

1. **Personalization:** The AI Roadmap Generator effectively creates unique study plans, solving the "one-size-fits-all" problem.
2. **Interactivity:** The integration of Judge0 allows users to apply what they learn immediately, solving the "passive learning" problem.
3. **Scalability:** The MERN stack architecture proved robust, handling concurrent requests efficiently during testing.

In conclusion, Reflect is not just a learning platform; it is an intelligent companion that evolves with the student. It bridges the gap between theoretical knowledge and practical application, providing a closed-loop system for mastering computer science concepts.

7.2 Future Work

While the current iteration of Reflect acts as a fully functional Minimum Viable Product (MVP), the roadmap for future development includes several exciting enhancements:

1. **Mobile Application:** Developing a dedicated React Native application to allow students to learn on the go, with offline capabilities for reading roadmap content.
2. **Voice Interaction:** Integrating OpenAI's Whisper model to allow users to ask questions verbally and receive audio responses, simulating a real conversation with a tutor.
3. **Multiplayer Coding Battles:** Introducing a real-time WebSocket-based module where two students can race to solve the same coding problem, adding a competitive social layer.
4. **Resume Builder:** An AI module that analyzes a student's completed roadmaps and quiz scores to automatically generate a PDF resume highlighting their verified skills.
5. **Job Matching:** Partnering with recruiters to match students with internships based on their proven proficiency in specific roadmap node.

8. References

1. **IEEE Computer Society.** (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998). New York, NY: Institute of Electrical and Electronics Engineers.
2. **Next.js Documentation.** (2024). *The React Framework for the Web*. Vercel. Retrieved from: <https://nextjs.org/docs>
3. **React.js Documentation.** (2024). *The library for web and native user interfaces*. Meta. Retrieved from: <https://react.dev/>
4. **Node.js Foundation.** (2024). *Node.js Documentation*. Retrieved from: <https://nodejs.org/en/docs/>
5. **MongoDB, Inc.** (2024). *MongoDB Manual: Schema Design Best Practices*. Retrieved from: <https://www.mongodb.com/docs/manual/core/data-models/>
6. **Groq Cloud.** (2024). *Groq API Reference & Documentation*. Retrieved from: <https://console.groq.com/docs>
7. **Judge0.** (2024). *Judge0 API Documentation: Open Source Online Code Execution System*. Retrieved from: <https://judge0.com/docs>
8. **Auth0.** (2024). *JSON Web Token (JWT) Introduction*. Retrieved from: <https://jwt.io/introduction>
9. **Sommerville, I.** (2011). *Software Engineering* (9th ed.). Boston, MA: Addison-Wesley.
10. **Pressman, R. S.** (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
11. **Ambler, S. W.** (2002). *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*. Wiley. Retrieved from: <http://agilemodeling.com/>
12. **Microsoft.** (2024). *REST API Design Best Practices*. Microsoft Learn. Retrieved from: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>
13. **World Wide Web Consortium (W3C).** (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved from: <https://www.w3.org/TR/WCAG21/>
14. **OpenAI.** (2024). *Platform Documentation & Prompt Engineering Guidelines*. Retrieved from: <https://platform.openai.com/docs>
15. **UNESCO.** (2023). *Artificial Intelligence in Education: Challenges and Opportunities for Sustainable Development*. Retrieved from: <https://unesdoc.unesco.org/>
16. **HolonIQ.** (2024). *Global EdTech Market Report: The Rise of AI Tutors*. Retrieved

from: <https://www.holoniq.com/>

17. **eLearning Industry.** (2024). *The Future of Adaptive Learning Systems*. Retrieved from: <https://elearningindustry.com/>
18. **Tailwind Labs.** (2024). *Tailwind CSS Documentation*. Retrieved from: <https://tailwindcss.com/docs>
19. **Mozilla Developer Network (MDN).** (2024). *Web Docs: HTTP, HTML, CSS, and JavaScript*. Retrieved from: <https://developer.mozilla.org/>
20. **European Union.** (2018). *General Data Protection Regulation (GDPR)*. Retrieved from: <https://gdpr.eu/>